



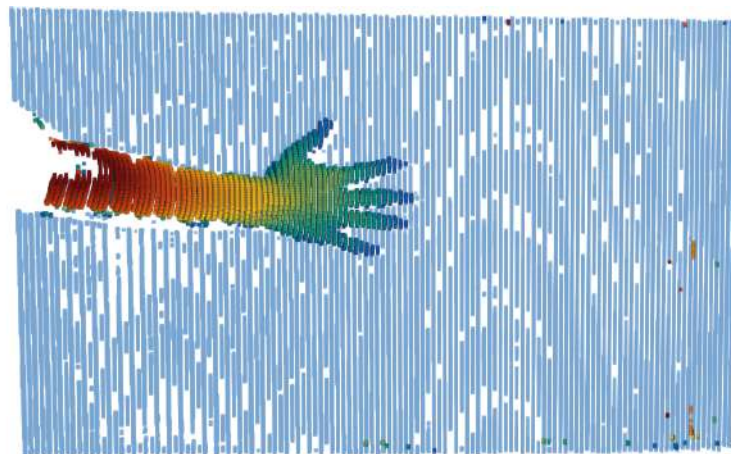
28 mai 2026

BOULARD Gaëtan, CORDIER Léandre, JULLIERE Thomas, RAMSIS Zainab,
RANNOU Mathieu

Sujet : Imageur 3D - **Nantes**

Groupe 1

UE Pronto - Rapport Final 2026



Bras imagé en 3D dans le cadre de notre projet



IMT Atlantique

Bretagne-Pays de la Loire
École Mines-Télécom

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Objectifs	4
1.3	Éléments à notre disposition	5
2	Fonctionnalités à développer	7
2.1	Acquisition automatique et séquençement des données	7
2.2	Traitement du signal et binarisation dynamique	8
2.3	Calibration automatique du système	8
3	Reconstruction Spatiale 3D par Lumière Structurée	8
3.1	Modélisation Géométrique et Étalonnage du Système	8
3.2	Décodage Temporel et Génération de la Matrice de Localisation	10
3.3	Identification Absolue de l'Abscisse Émetteur	10
3.4	Triangulation Spatiale par Résolution du Système Linéaire	11
3.5	Analyse Algorithmique des Instabilités de Reconstruction et Corrections Émergentes	12
3.6	Interface Graphique Utilisateur (IHM)	15
3.7	Sauvegarde et Exportabilité des données	16
4	Conception	17
4.1	Architecture du code	17
4.2	Interface graphique	18
4.3	Objet support	19
5	Développement	20
5.1	Interface graphique	20
5.2	Cache image	20
6	Intégration et validation	21
6.1	Interface graphique	21
6.2	Validation de la calibration	21
6.3	Validation de la reconstruction	27
7	Clôture du projet	32
7.1	Tableau des rendus techniques/scientifiques	32
7.2	Validation	32
7.2.1	Calibration	32
7.2.2	Acquisition	33
7.3	Axes d'amélioration du système	34
8	Retour d'expérience	35

8.1	Retour d'expérience sur la gestion de projet	35
8.2	Retour d'expérience sur le travail en équipe	36
9	Annexes	36
9.1	Bibliographie	36
9.2	Bonus : gestion de la couleur	37
9.3	Codes	38
9.4	Retours d'expériences individuels	41

1 Introduction

1.1 Contexte

Le domaine de la métrologie optique a été révolutionné par l'imagerie par illumination structurée, une technique de mesure 3D sans contact permettant de modéliser des surfaces avec une précision et une rapidité croissantes. L'objectif de ce projet est de concevoir un imageur 3D dédié aux objets de grandes dimensions, allant de 1 à 4 m², afin de répondre aux besoins de secteurs industriels tels que l'aéronautique ou le nautisme.

Pour ce faire, nous utilisons la technologie binaire, qui présente l'avantage de réduire considérablement le nombre d'images nécessaires à la reconstruction par rapport aux méthodes classiques. Ce choix technique permet de minimiser les capacités de stockage requises et d'accélérer le traitement informatique des données. Enfin, notre démarche s'inscrit dans une logique de faible coût, en utilisant des composants grand public comme un vidéoprojecteur et une caméra standard, nous visons une résolution de 1 mm.

La technologie retenue est donc une **imagerie 3D par lumière structurée à franges binaires**. Le vidéoprojecteur joue le rôle d'émetteur en envoyant sur l'objet une succession de motifs noir et blanc, dont les franges codent spatialement les positions du projecteur. La caméra observe la déformation de ces franges sur la surface mesurée. Après seuillage des images capturées, chaque pixel caméra est associé à un code binaire, ce qui permet de retrouver la frange projetée correspondante puis de reconstruire la position 3D par triangulation.

Le principe consiste à projeter successivement une série de N trames binaires depuis le vidéoprojecteur. Chaque trame possède une fréquence spatiale croissante, permettant d'identifier de manière unique chaque position éclairée sur l'objet.

Après une étape préalable de calibration du système projecteur-caméra, chaque point observé par la caméra reçoit une succession d'intensités lumineuses (blanc ou noir) au cours des différentes projections. La succession de ces états forme alors un **mot binaire** permettant d'identifier la frange incidente associée à ce point.

Ainsi, pour N projections binaires, il devient possible de coder jusqu'à 2^N positions distinctes du projecteur. La correspondance entre pixels caméra et pixels projecteur peut alors être établie, permettant la reconstruction tridimensionnelle par triangulation.

1.2 Objectifs

L'objectif de ce projet est l'étude et la réalisation d'un imageur tridimensionnel pour objets de grandes tailles. Le projet sera réalisé en deux étapes : dans un premier temps nous avons fait une étude théorique ainsi qu'une simulation numérique afin d'établir les relations mathématiques et de créer les codes de traitement des images. Puis nous avons réalisé un système physique (avec un projecteur et une caméra), et créé une interface graphique. Nous voulions aussi imprimer en 3D l'objet reconstruit à échelle réduite.

Notre capteur doit être en mesure de mesurer de grands objets de 1 à 4 m² avec une résolution minimale de 1 mm. De plus, la mesure se fera sans contact, avec la projection d'une série d'image sur un objet, et la mesure des rayons réfléchis. Enfin, la calibration du capteur devra se faire de manière automatique à partir d'images prises sur un damier.

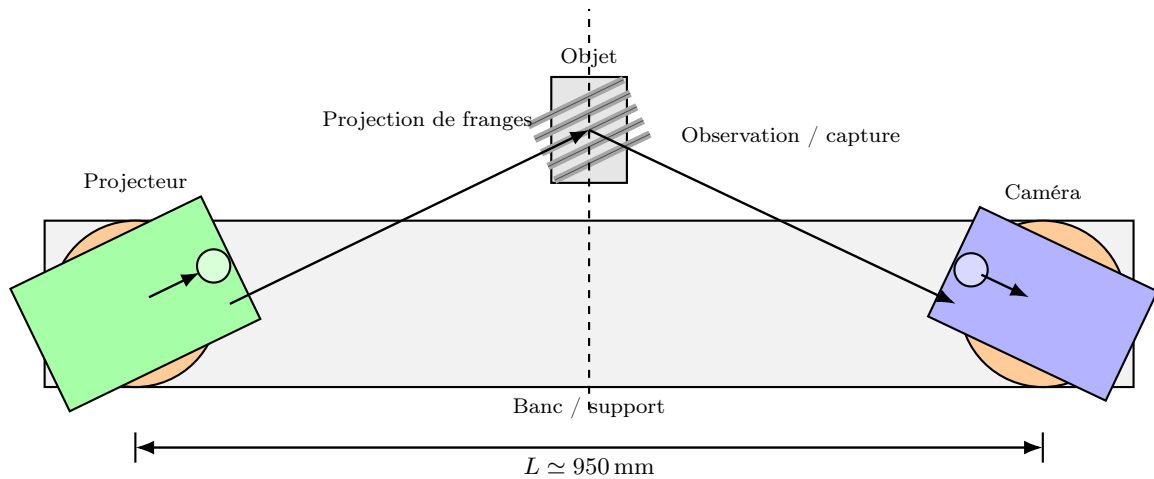


FIGURE 1 – Vue de haut : projecteur et caméra orientés vers l'objet, franges projetées (schématique)

1.3 Éléments à notre disposition

L'émetteur et le récepteur nous ont été fournis, la caméra est une Webcam C920E [1] et le projecteur est un EPSON EB-W49[2]. Au début du projet nous avons des éléments laissés par les équipes des années précédentes tels que :

- Un support pour l'émetteur et le récepteur avec deux parties tournantes. Du côté émetteur, la hauteur du support peut être modifiée avec trois pieds réglables. Une roue avec des mesures d'angles est placée sous la caméra, ce qui nous permet d'avoir un ordre de grandeur de l'angle de la caméra. Côté projecteur, l'inclinaison est réglable grâce à un boulon à l'arrière de la planche supportant l'émetteur.



FIGURE 2 – Projecteur sur le support



FIGURE 3 – Caméra sur le support

- Des éléments de volumes en polystyrène extrudé nous étaient également fournis, notamment un plan avec un damier noir et blanc utile à la calibration, et d'autres volumes comme une pyramide ou le logo des Écoles des Mines, utiles afin de tester les capacités de reconstruction de notre projet.

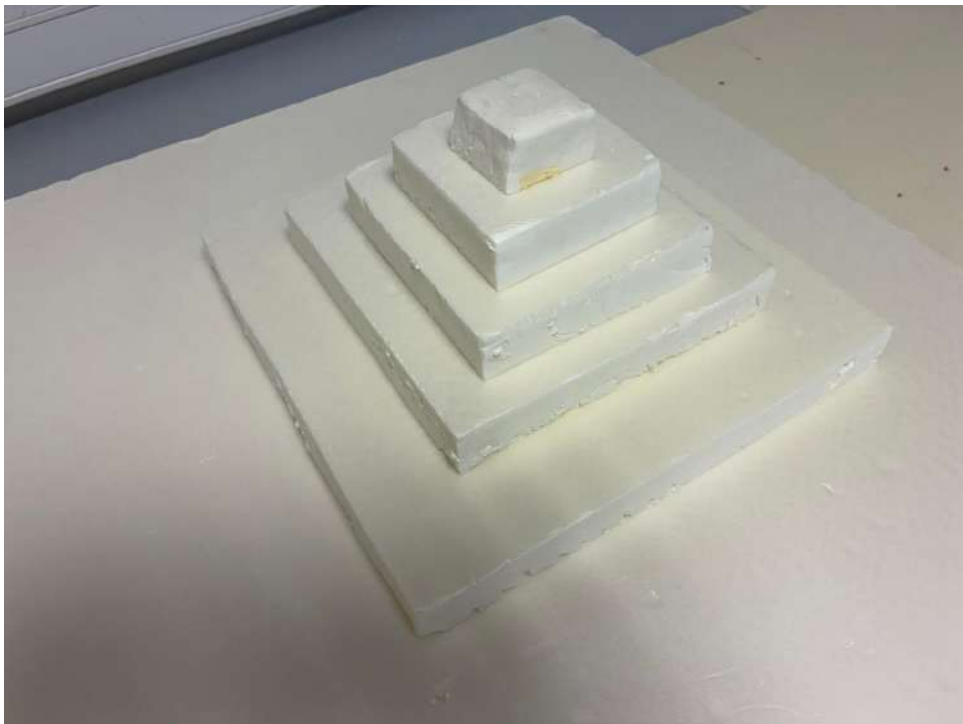


FIGURE 4 – Pyramide en polystyrène



FIGURE 5 – Logo des Écoles des Mines

2 Fonctionnalités à développer

Le système d'imagerie 3D par illumination structurée repose sur un enchaînement rigoureux de fonctionnalités logicielles et matérielles. L'objectif principal est de piloter l'acquisition, d'assurer une transformation exacte des coordonnées pixels vers l'espace métrique, et de restituer un modèle 3D exploitable par l'utilisateur.

2.1 Acquisition automatique et séquençage des données

Le processus d'acquisition doit garantir la parfaite synchronisation entre le vidéoprojecteur (émetteur) et la webcam (récepteur).

- **Génération et Projection des Trames :** Le système génère de manière automatique une séquence de N trames binaires. Dans le protocole final, nous avons travaillé avec 8 trames projetées, puis retenu les 7 premières pour la reconstruction car la dernière contenait des franges trop fines pour être détectée de manière fiable. Chaque image se présente sous la forme d'une matrice binaire de résolution native 1280×800 pixels (propre au vidéoprojecteur EPSON EB-W49). Ces trames, exportées au format PNG 8-bit non compressé pour éviter tout flou de transition sur les franges, sont transmises en plein écran via une liaison HDMI.
- **Capture et Contrôle Caméra :** Parallèlement à la projection, la webcam Logitech C920e capture la réflexion des franges sur l'objet. Afin d'éviter les biais d'exposition ou de mise au point automatique qui dégraderaient le traitement, les paramètres de capture (temps d'exposition, gain, balance des blancs) sont entièrement verrouillés et pilotés via la bibliothèque OpenCV. Les images acquises en résolution 1920×1080 pixels sont instantanément vectorisées sous forme de tenseurs multidimensionnels pour optimiser la vitesse de traitement.

2.2 Traitement du signal et binarisation dynamique

Pour passer de l'image brute à une matrice logique exploitable, les fonctionnalités suivantes de nettoyage des données sont appliquées :

- **Filtrage du bruit** : Application d'un filtre spatial passe-bas pour éliminer le bruit haute fréquence inhérent au capteur de la caméra.
- **Seuillage adaptatif** : Afin de contrer les variations d'albédo de l'objet ou la dispersion de l'énergie lumineuse due au recul important du projecteur, le logiciel calcule dynamiquement un seuil optimal (méthode de maximisation de la variance inter-classe). Cela permet de séparer fidèlement les zones éclairées (1) des zones sombres (0), même sur des objets de géométrie complexe ou de couleur sombre.
- **Masque manuel** Afin de faciliter le traitement des images via nos codes, nous avons choisi d'appliquer manuellement un masque autour de celles-ci. Cette étape est détaillée ici

2.3 Calibration automatique du système

La calibration est la brique logicielle critique permettant de lier les coordonnées bidimensionnelles des capteurs aux mesures métriques réelles.

- **Calibration du Récepteur** : À partir de la capture d'un damier de référence positionné à des profondeurs connues, l'algorithme détecte les points d'intersection. En résolvant un système d'équations homogènes s'appuyant sur au moins 6 points de référence, le programme extrait la matrice de projection perspective normalisée du récepteur (M_{RN}), modélisant ses paramètres intrinsèques et extrinsèques.
- **Calibration de l'Émetteur** : À l'aide d'une méthode indirecte, le projecteur affiche une mire numérique sur le plan du damier. Les points identifiés par la caméra permettent, par intersection de rayons optiques et triangulation avec le plan physique, de calculer la matrice de projection perspective normalisée de l'émetteur (M_{EN}).

3 Reconstruction Spatiale 3D par Lumière Structurée

La reconstruction géométrique tridimensionnelle d'un objet d'intérêt à partir d'un système couplé caméra-projecteur repose sur l'inversion d'un modèle de projection perspective préalablement étalonné. Le formalisme mathématique exposé ci-après décrit le passage exhaustif des espaces de codage bidimensionnels discrets à un nuage de points métrique caractérisant la topologie de la surface observée.

3.1 Modélisation Géométrique et Étalonnage du Système

Le changement de repère des coordonnées pixéliques de l'espace image vers l'espace cartésien euclidien réel ($\mathbb{R}^3$) nécessite l'estimation des matrices de projection perspective normalisées. Ces opérateurs modélisent les transformations géométriques intrinsèques et extrinsèques régissant le récepteur (caméra) et l'émetteur (projecteur).

Le processus d'étalonnage débute par la caractérisation de l'appareil de capture optique. Une mire bidimensionnelle de géométrie connue (typiquement un motif en damier, illustré en Figure 6) est positionnée à une

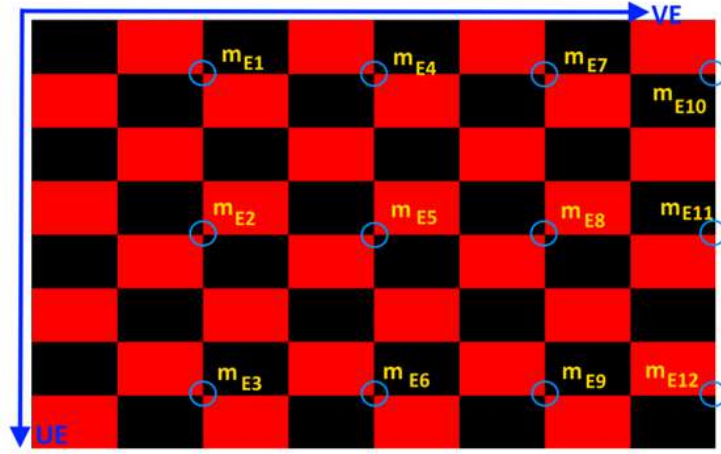


FIGURE 6 – Damier

profondeur de référence Z_i .

Pour un ensemble de n points de référence cartésiens $M_i = [X_i, Y_i, Z_i]^T \in \mathbb{R}^3$ (où $n \geq 6$), nous mesurons l'ensemble des projections correspondantes $m_{Ri} = [u_{Ri}, v_{Ri}]^T$ au sein de la matrice du capteur CCD. L'opérateur de projection du récepteur, noté $M_{RN} \in \mathbb{R}^{3 \times 4}$, est modélisé par le système d'équations homogènes suivant :

$$s \begin{pmatrix} u_{Ri} \\ v_{Ri} \\ 1 \end{pmatrix} = M_{RN} \begin{pmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{pmatrix} \quad (1)$$

Dans l'expression (1), le scalaire $s \in \mathbb{R}^*$ désigne le facteur d'échelle projective, structurellement proportionnel à la profondeur locale du point par rapport au centre optique du récepteur, pondéré par sa distance focale f_R . La matrice M_{RN} dispose de 12 coefficients définis à un facteur multiplicatif près, restreignant l'espace de recherche à 11 degrés de liberté indépendants. L'extraction de ces paramètres nécessite ainsi un minimum de 6 points non coplanaires fournissant un système linéaire déterminé de 12 équations.

L'étalonnage de l'émetteur est résolu par une approche duale indirecte. La projection d'une mire numérique synthétique engendre un ensemble de points de coordonnées connues $m_{Ej} = [u_{Ej}, v_{Ej}]^T$ issus du plan LCD. Ces motifs sont projetés sur la scène et acquis par le récepteur sous forme de points images $m_{Rj} = [u_{Rj}, v_{Rj}]^T$. L'intersection géométrique des rayons optiques rétro-projetés issus de la caméra avec le plan d'appui fournit les coordonnées spatiales 3D des points réfléchis. La matrice de projection de l'émetteur $M_{EN} \in \mathbb{R}^{3 \times 4}$ est dès lors calculée via la relation :

$$s' \begin{pmatrix} u_{Ej} \\ v_{Ej} \\ 1 \end{pmatrix} = M_{EN} \begin{pmatrix} X_j \\ Y_j \\ Z_j \\ 1 \end{pmatrix} \quad (2)$$

L'amplitude du facteur d'échelle s' est explicitée analytiquement par le produit de la focale de l'émetteur f_E et de sa constante d'échelle de numérisation σ_E , se simplifiant au niveau de la surface de référence par la coordonnée de profondeur :

$$s' = f_E \cdot \sigma_E = Z_i \quad (3)$$

3.2 Décodage Temporel et Génération de la Matrice de Localisation

Afin d'établir l'appariement pixel à pixel entre le plan du capteur de capture et le plan de projection, une séquence de N franges binaires est projetée. Soit $M_k(u_R, v_R) \in \{0, 1\}$ l'état binaire du pixel (u_R, v_R) extrait lors de la k -ième capture. Le code binaire absolu $C(u_R, v_R) \in \mathbb{N}$ est sommé selon la fonction d'ordre pondérée suivante :

$$C(u_R, v_R) = \sum_{k=0}^{N-1} M_k(u_R, v_R) \cdot 2^{N-1-k} \quad (4)$$

Cette convolution temporelle discrète projette l'ensemble des images 2D vers une matrice d'état de localisation, notée L_C , où chaque cellule contient l'identifiant de la frange émise par le projecteur. L'implémentation algorithmique de cette phase de binarisation et de décodage est présentée dans le Listing 1.

```

1 def binariser_franges(captures: np.ndarray, masque: np.ndarray) -> np.ndarray:
2     # Analyse locale du contraste pour pallier les défauts d'un seuil global
3     image_max = captures.max(axis=0).astype(np.float32)
4     image_min = captures.min(axis=0).astype(np.float32)
5     seuil = (image_max + image_min) * 0.5
6     # Traduction directe du predicat d'activation binaire
7     binaire = (captures.astype(np.float32) > seuil[None, :, :]).astype(np.uint8)
8     binaire[:, ~masque] = 0
9     return binaire
10
11 def decoder_coordonnees_projecteur(franges_binaires: np.ndarray, masque: np.ndarray) -> np.
    ndarray:
12     # Inversion de phase en accord avec la structure des bandes du generateur
13     franges_inversees = 1 - franges_binaires
14     nombre_bits = franges_inversees.shape[0]
15     # Generation du vecteur de ponderation de l'equation (4)
16     poids = (2 ** np.arange(nombre_bits - 1, -1, -1)).astype(np.int32)[:, None, None]
17     code = np.sum(franges_inversees * poids, axis=0)
18     code[~masque] = 0
19     # Filtrage median pour eliminer le bruit de quantification aux transitions
20     code = cv2.medianBlur(code.astype(np.uint16), 3).astype(np.float32)
21     code[~masque] = 0
22     return code

```

Listing 1 – Algorithme de binarisation dynamique locale et de décodage des codes de franges.

3.3 Identification Absolue de l'Abscisse Émetteur

L'évaluation de l'indice de frange $C(u_R, v_R)$ permet d'isoler de manière univoque la coordonnée horizontale discrète v_E caractérisant le plan LCD d'émission. En désignant par v_{Eg} et v_{Ed} les frontières spatiales gauche et droite de la frange considérée, la coordonnée centrale normalisée s'écrit :

$$v_E = \frac{v_{Eg} + v_{Ed}}{2} \quad \text{avec} \quad v_{Eg} = \left(\frac{N_{bHE}}{2^N} \cdot C \right) + 1 \quad (5)$$

Où N_{bHE} matérialise la dimension de la largeur physique du projecteur en pixels. Au sein du pipeline de calcul, cette discrétisation géométrique se traduit par le calcul du pas d'échantillonnage de la lumière structurée :

$$\Delta_{px} = \frac{\text{LARGEUR_PROJECTEUR_PX}}{2^N} \quad (6)$$

3.4 Triangulation Spatiale par Résolution du Système Linéaire

À l'issue de la phase d'extraction, chaque pixel d'intérêt du capteur est associé à un triplet d'observations couplées (u_R, v_R, v_E) . L'estimation des coordonnées réelles tridimensionnelles $P = [X, Y, Z]^T \in \mathbb{R}^3$ revient à résoudre un système surdéterminé de trois équations algébriques indépendantes à trois inconnues. En décomposant les matrices de projection calibrées du récepteur ($M_{RN} = \{m_{ijR}\}$) et de l'émetteur ($M_{EN} = \{m'_{ijE}\}$), le système linéaire s'énonce tel que :

$$\begin{cases} (m_{11R} - u_R m_{31R})X + (m_{12R} - u_R m_{32R})Y + (m_{13R} - u_R m_{33R})Z \\ \quad = u_R m_{34R} - m_{14R} \\ (m_{21R} - v_R m_{31R})X + (m_{22R} - v_R m_{32R})Y + (m_{23R} - v_R m_{33R})Z \\ \quad = v_R m_{34R} - m_{24R} \\ (m'_{11E} - v_E m'_{31E})X + (m'_{12E} - v_E m'_{32E})Y + (m'_{13E} - v_E m'_{33E})Z \\ \quad = v_E m'_{34E} - m'_{14E} \end{cases} \quad (7)$$

Ce système d'équations (7) est résolu par décomposition algébrique linéaire en appliquant la méthode des moindres carrés pour chaque pixel validé par le masque photométrique. L'implémentation de cette étape critique de triangulation numérique est détaillée dans le Listing 2.

```

1 def trianguler(
2     code_projecteur: np.ndarray,
3     masque_objet: np.ndarray,
4     matrice_recepteur: np.ndarray, # Correspond a la matrice MRN
5     matrice_emetteur: np.ndarray, # Correspond a la matrice MEN
6     pas_projecteur: float,         # Pas spatial associe a l'equation (5)
7 ) -> np.ndarray:
8     hauteur, largeur = code_projecteur.shape
9     lignes, colonnes = np.indices((hauteur, largeur), dtype=np.float64)
10    valide = masque_objet
11
12    u_recepteur = colonnes[valide]
13    v_recepteur = lignes[valide]
14    # Calcul de l'abscisse discrete du projecteur vE de l'equation (5)
15    x_projecteur = (code_projecteur[valide] + 0.5) * pas_projecteur
16
17    nombre_points = len(u_recepteur)
18    systeme = np.zeros((nombre_points, 3, 3), dtype=np.float64)
19    second_membre = np.zeros((nombre_points, 3), dtype=np.float64)
    
```

```
20
21 # Formulation de la matrice A du systeme lineaire lie a l'equation (6)
22 systeme[:, 0, :] = u_recepteur[:, None] * matrice_recepteur[2, :3] - matrice_recepteur[0, :3]
23 systeme[:, 1, :] = v_recepteur[:, None] * matrice_recepteur[2, :3] - matrice_recepteur[1, :3]
24 systeme[:, 2, :] = x_projecteur[:, None] * matrice_emetteur[2, :3] - matrice_emetteur[0, :3]
25
26 # Formulation du second membre B du vecteur de projection
27 second_membre[:, 0] = matrice_recepteur[0, 3] - u_recepteur * matrice_recepteur[2, 3]
28 second_membre[:, 1] = matrice_recepteur[1, 3] - v_recepteur * matrice_recepteur[2, 3]
29 second_membre[:, 2] = matrice_emetteur[0, 3] - x_projecteur * matrice_emetteur[2, 3]
30
31 # Resolution numerique directe via solveur SVD/LU pour chaque point
32 points = np.linalg.solve(systeme, second_membre[:, None]).reshape(nombre_points, 3)
33 return points[np.isfinite(points).all(axis=1)]
```

Listing 2 – Traduction matricielle et resolution par moindres carres du systeme de triangulation.

La résolution de ce système pour l'ensemble du domaine d'intérêt génère un nuage de points tridimensionnel brut, qui subit ensuite un changement de base pour s'exprimer dans le repère métrique de la table d'appui via une décomposition en valeurs singulières (SVD) du plan de référence. Cette opération finale conclut rigoureusement la phase d'acquisition et de reconstruction spatiale.

3.5 Analyse Algorithmique des Instabilités de Reconstruction et Corrections Émergentes

L'implémentation naïve des modèles projectifs exposés à la Section 3.1 confronte le pipeline numérique à d'importantes divergences topologiques. L'analyse des premiers essais de reconstruction de la mire géométrique (la pyramide) met en évidence deux sources majeures d'artefacts : l'absence de discrimination spectrale du fond ambiant et l'incapacité d'un seuil global à absorber la réflectance non uniforme de l'objet, induisant des aberrations de calcul de plusieurs ordres de grandeur (voir Figure 8).

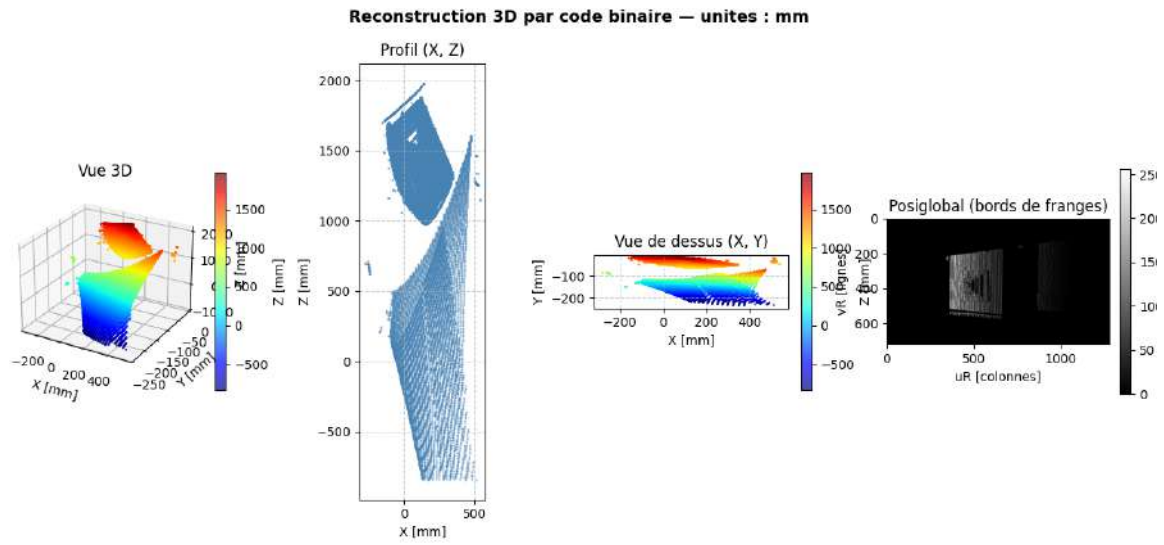


FIGURE 8 – Reconstruction avec déformations

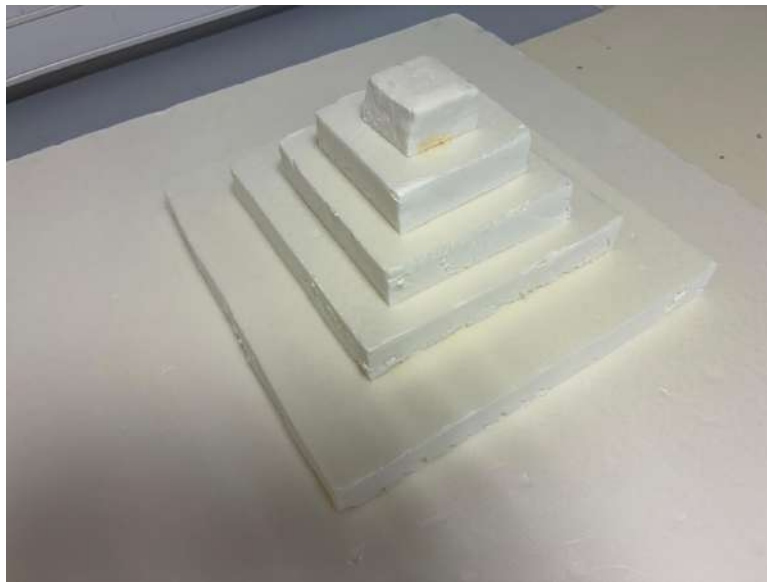


FIGURE 7 – Image de l'objet de base

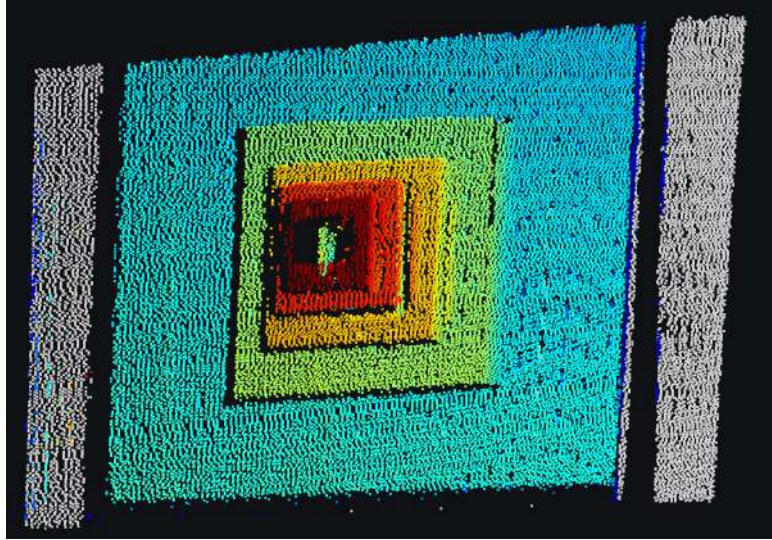


FIGURE 9 – Reconstruction corrigée

TABLE 1 – Analyse des Hypothèses d’Erreur et Correctifs Associés

Source d’Erreur	Hypothèse / Mécanisme Physique	Formalisation du Correctif
Bruit de fond hors modulation	Éclairement ambiant constant assimilé à tort à un code binaire valide par Otsu global.	Critère de modulation local : $\Delta_{\mathcal{M}}(u, v) > \tau_{\mathcal{M}}$ avec $\Delta_{\mathcal{M}} = \max_k(M_k) - \min_k(M_k)$.
Distorsion et non-uniformité	Atténuation spatiale de l’intensité lumineuse et distorsion radiale de l’optique réceptrice.	Seuil dynamique pixélique : $\mu(u, v) = \frac{1}{N} \sum_{k=0}^{N-1} M_k(u, v)$. Binarisation adaptative.
Points aberrants résiduels	Transitions de franges bruitées (queues de distribution lourdes sur Z).	Double filtrage statistique robuste : MAD (5σ) suivi d’un fenêtrage par Intervalle Interquartile (IQR = 1.5).

Le Tableau 1 synthétise les hypothèses physiques des sources d’erreurs identifiées ainsi que les stratégies mathématiques de régularisation retenues pour valider le pipeline d’acquisition 2026.

Le Listing 3 illustre de manière compacte le basculement algorithmique entre l’approche globale instable et l’implémentation locale retenue pour assurer la convergence de la triangulation de Faugeras-Toscani.

```

1 # =====
2 # PARADIGME INITIAL INSTABLE (Seuillage global et absence de masque)
3 # =====
4 # Erreur : Otsu traite le bruit de fond, generant des codes aberrants
5 seuil_global = 120
6 binaire_naif = (captures_brutes > seuil_global).astype(np.uint8)
7 code_instable = np.sum(binaire_naif * poids, axis=2)

```

```

8
9 # =====
10 # PARADIGME PROPAGE ET CORRIGE (Filtrage de modulation et seuil local)
11 # =====
12 # Correctif 1 & 2 : Calcul de la modulation locale et du seuil moyen
13 modulation = captures.max(axis=2) - captures.min(axis=2)
14 seuil_local = captures.mean(axis=2) # Palliatif d'atténuation optique
15
16 masque_valide = modulation > 30 # Elimination stricte du bruit de fond
17 IRbin = (captures > seuil_local[:, :, None]).astype(np.int32)
18 IRbin[~masque_valide] = 0 # Rejection des pixels non éclairés
19
20 # Correctif 3 : Post-filtrage statistique sur le nuage de points Z
21 z_med = np.median(XYZ[:, 2])
22 z_mad = np.median(np.abs(XYZ[:, 2] - z_med))
23 ok = np.abs(XYZ[:, 2] - z_med) < max(5.0 * 1.4826 * z_mad, 250.0)
24 XYZ_filtre = XYZ[ok] # Rejection finale par IQR standard (facteur 1.5)

```

Listing 3 – Mise en parallèle des paradigmes de traitement d’images (Global vs. Local).

Cette restructuration algorithmique élimine l’étalement non physique des coordonnées projetées, garantissant que 100% des vecteurs calculés s’inscrivent dans la géométrie réelle de l’espace métrique d’acquisition.

3.6 Interface Graphique Utilisateur (IHM)

Pour rendre le système maniable et masquer la complexité mathématique sous-jacente, l’interface graphique en C# utilisant le framework .NET centralise les fonctionnalités utilisateur suivantes :

- **Retour vidéo en temps réel** : Permet à l’opérateur d’ajuster le cadrage, de vérifier l’éclairage ambiant et de s’assurer de la bonne couverture du champ de projection.
- **Automatisation et Suivi** : Un panneau de configuration permet de définir le nombre d’images, de lancer séquentiellement la calibration ou la mesure en un seul clic, tout en affichant une barre de progression claire de l’acquisition.
- **Gestion des erreurs** : En cas de mauvaise détection du damier ou de contraste insuffisant des franges, l’interface lève une alerte explicite guidant l’utilisateur pour corriger le problème.
- **Visualisation 3D interactive** : Intégration d’un module d’affichage tridimensionnel permettant de manipuler le nuage de points généré (fonctions de rotation, translation et zoom) avec application éventuelle d’une texture ou d’un codage couleur selon le relief.

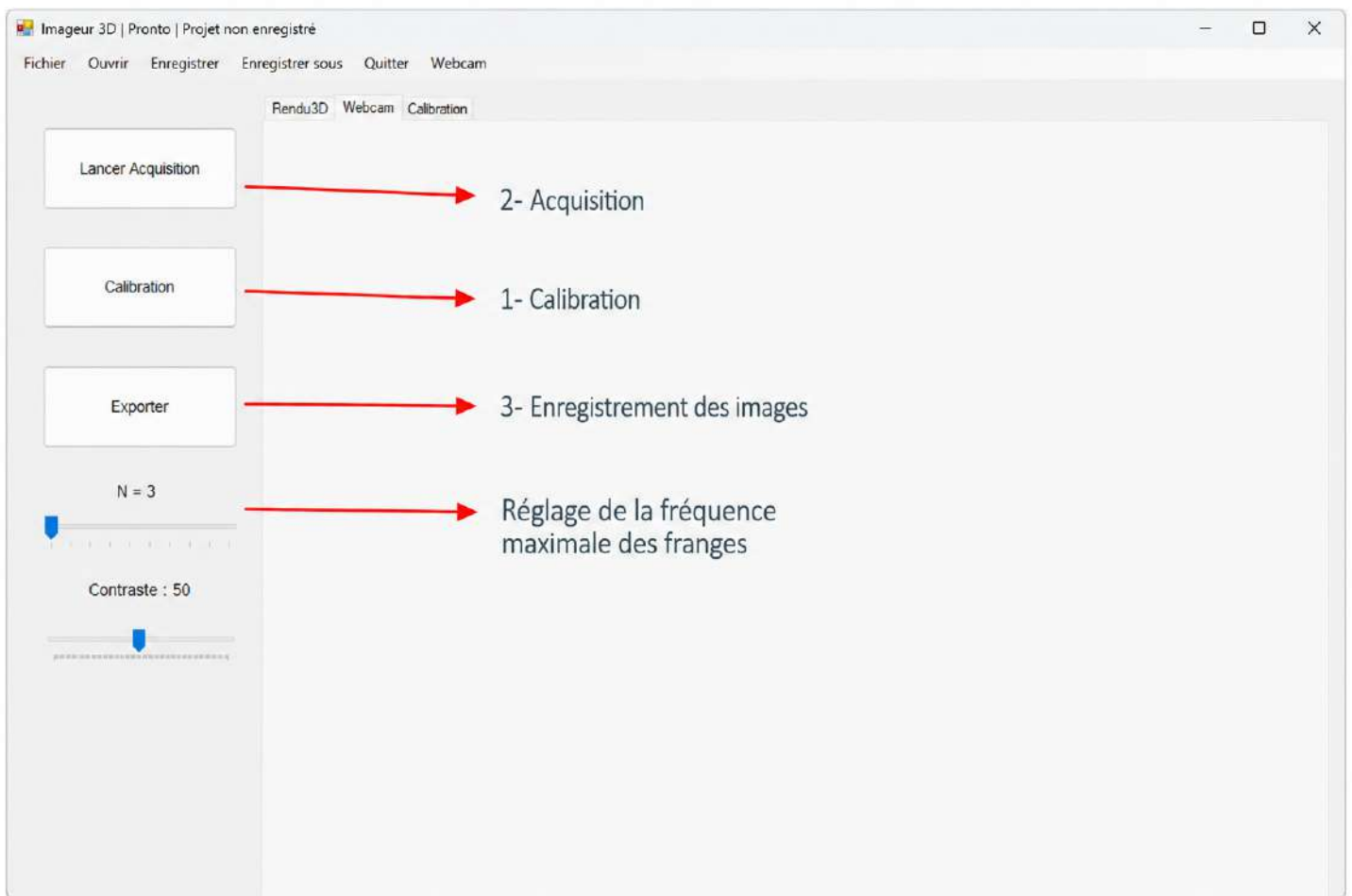


FIGURE 10 – Capture d'écran de l'interface graphique

3.7 Sauvegarde et Exportabilité des données

Le logiciel intègre des fonctions de gestion de fichiers garantissant la reproductibilité des mesures :

- **Sauvegarde de la configuration :** Exportation des coefficients des matrices de calibration (M_{RN} et M_{EN}) dans un format de fichier léger (ex : texte ou XML), permettant à l'utilisateur de charger une calibration existante lors de mesures ultérieures sans répéter le processus de ciblage.
- **Export du modèle 3D :** Restitue le nuage de points reconstruit sous forme de fichier de points et de page HTML interactive. Ces exports correspondent au format réellement utilisé pour analyser les résultats présentés dans ce rapport ; un export vers des formats de CAO comme .PLY ou .OBJ resterait une extension possible.

4 Conception

4.1 Architecture du code

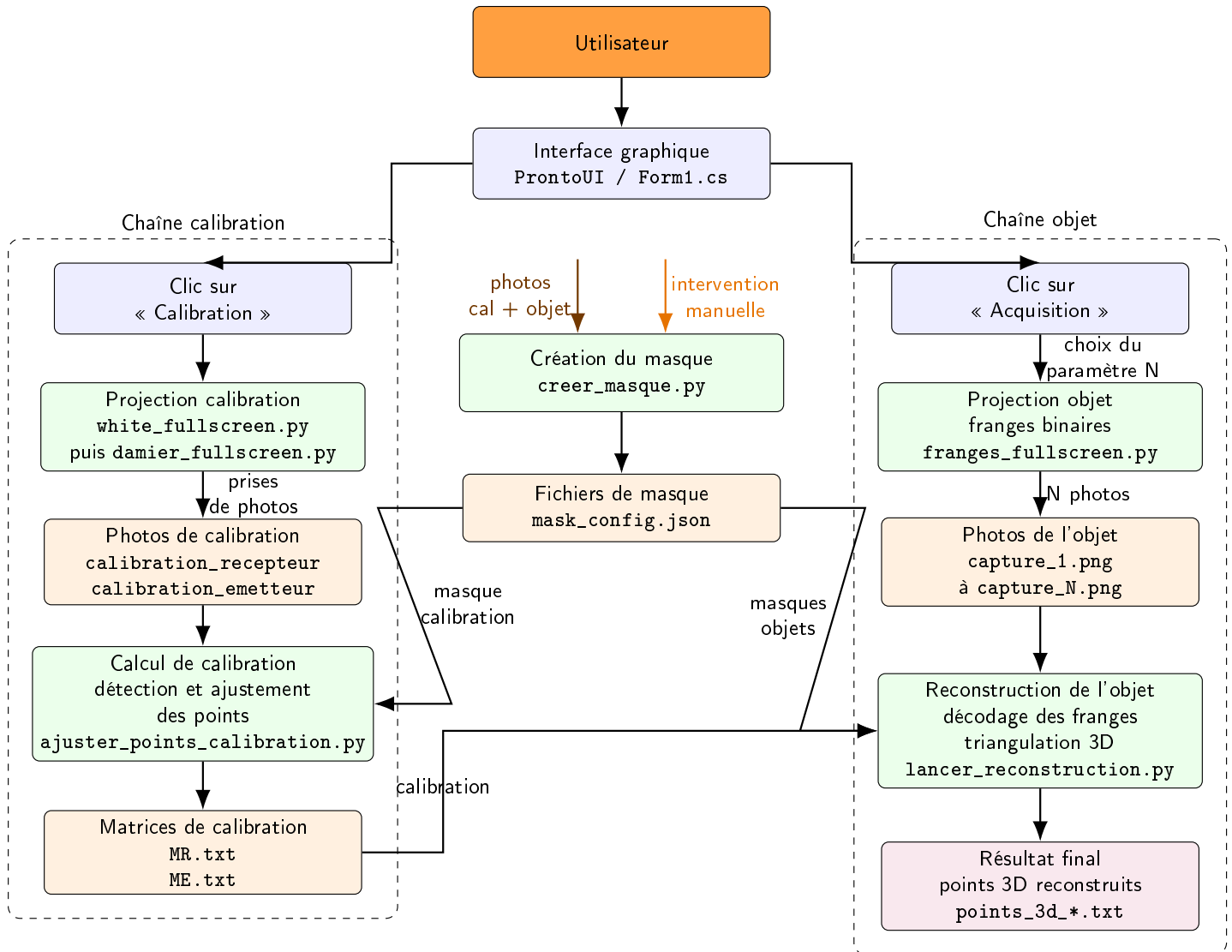


FIGURE 11 – Architecture générale de notre système complet de reconstruction d'objets

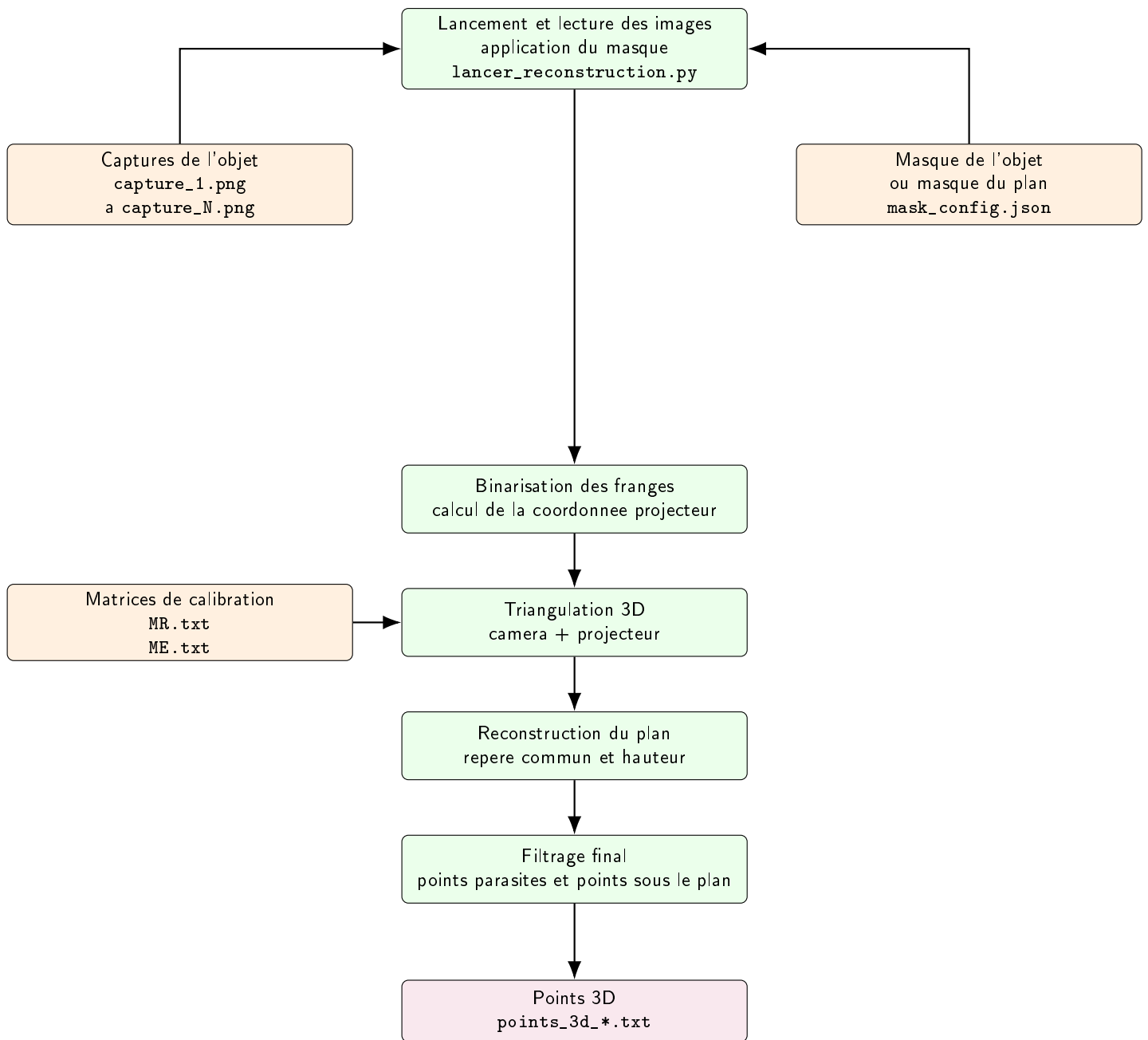


FIGURE 12 – Détail spécifique de l'architecture du code de reconstruction

4.2 Interface graphique

L'objectif de la création de l'interface graphique est de simplifier au maximum les différentes étapes nécessaires à la modélisation d'un objet en étant le plus lisible possible pour l'utilisateur. Nous avons donc d'abord déterminé ce dont l'utilisateur avait besoin pour suivre les étapes de reconstruction de l'objet :

- L'utilisateur doit avoir un retour de la caméra pour être capable de cadrer l'objet.
- Il doit pouvoir lancer la calibration manuellement et avoir un retour sur le fonctionnement.
- L'utilisateur doit être capable de voir la reconstruction de l'objet.

- Il doit pouvoir déterminer manuellement le nombre d'images à prendre.
- L'utilisateur doit pouvoir enregistrer les résultats facilement.

Nous avons choisi de coder l'interface en C# en utilisant le framework .NET sur visual studio car le framework est simple d'utilisation et qu'un membre de notre équipe l'avait déjà utilisé.

4.3 Objet support

L'objectif de la création du support est de simplifier au maximum les différentes étapes nécessaires à la calibration récepteur et émetteur. Nous avons donc d'abord déterminé ce dont l'utilisateur avait besoin pour effectuer les étapes de calibration de l'objet :

- L'insertion du damier dans le support doit être simple et rapide.
- Une fois inséré, le support doit être parfaitement stable.
- L'écart entre les deux positions du support doit être exactement de 10 cm.

Nous avons choisi de réaliser le support en utilisant le logiciel fusion 360, pour pouvoir ensuite l'imprimer en 3D.

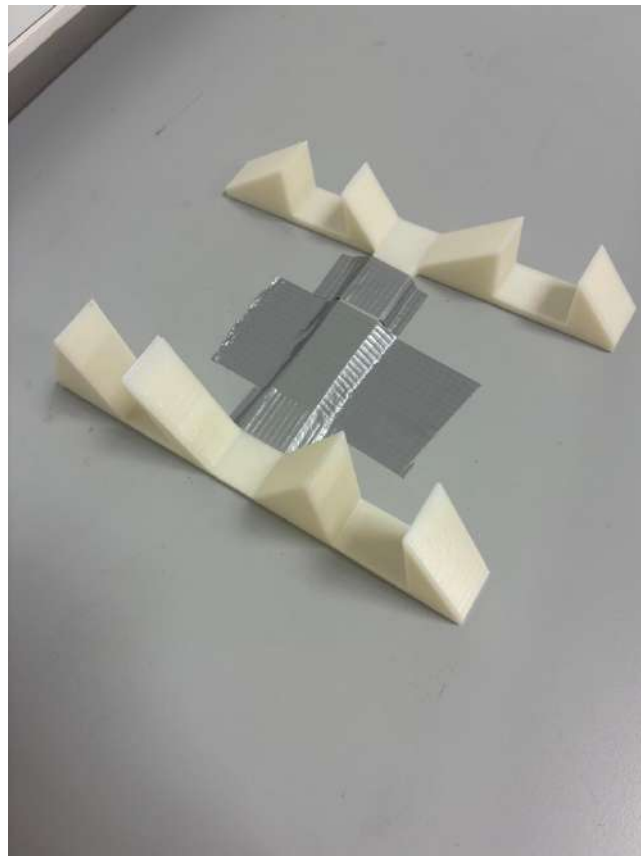


FIGURE 13 – Image d'un des supports imprimé

5 Développement

5.1 Interface graphique

Nous avons commencé à faire l'interface graphique assez tôt car nous terminions en parallèle les codes de reconstruction d'objet et de calibration . Lors du développement nous avons fait certains choix, nous avons mis un slider pour que l'utilisateur puisse choisir le nombre d'image à prendre simplement.

Lors de nos premiers essais nous avons eu des difficultés avec la caméra car du scintillement apparaissait sur les images de la caméra, nous avons donc rajouter du codes pour avoir accès aux paramètres de la caméra depuis l'interface graphique, ce qui nous permet d'ajuster les paramètres de la caméra grâce au retour avant de commencer les expérimentations.

Nous avons aussi fait des programmes Python pour générer des images de franges à projeter avec l'émetteur, et avons implémenté dans l'interface la projection automatique de ces images pour accélérer la capture des images. Pour enregistrer les différentes images nous avons choisi d'utiliser des dossiers zip.

5.2 Cache image

L'un des problème que nous avons rencontré est la présence d'éléments inutiles sur les photos (les éléments en dehors du damier pour la calibration, ou en dehors des rayures pour les mesures d'objets) ; ce qui engendrait des perturbations puisque notre programme essayait de traiter ces zones inutiles. Pour remédier à cette difficulté, nous avons mis en place un système de cache. Il s'agit de sélectionner à la main un rectangle contenant les éléments que nous voulons traiter (voir image ci-dessous), et notre code homogénéisera la zone extérieure au cadre en noir pour qu'elle ne perturbe plus le processus.

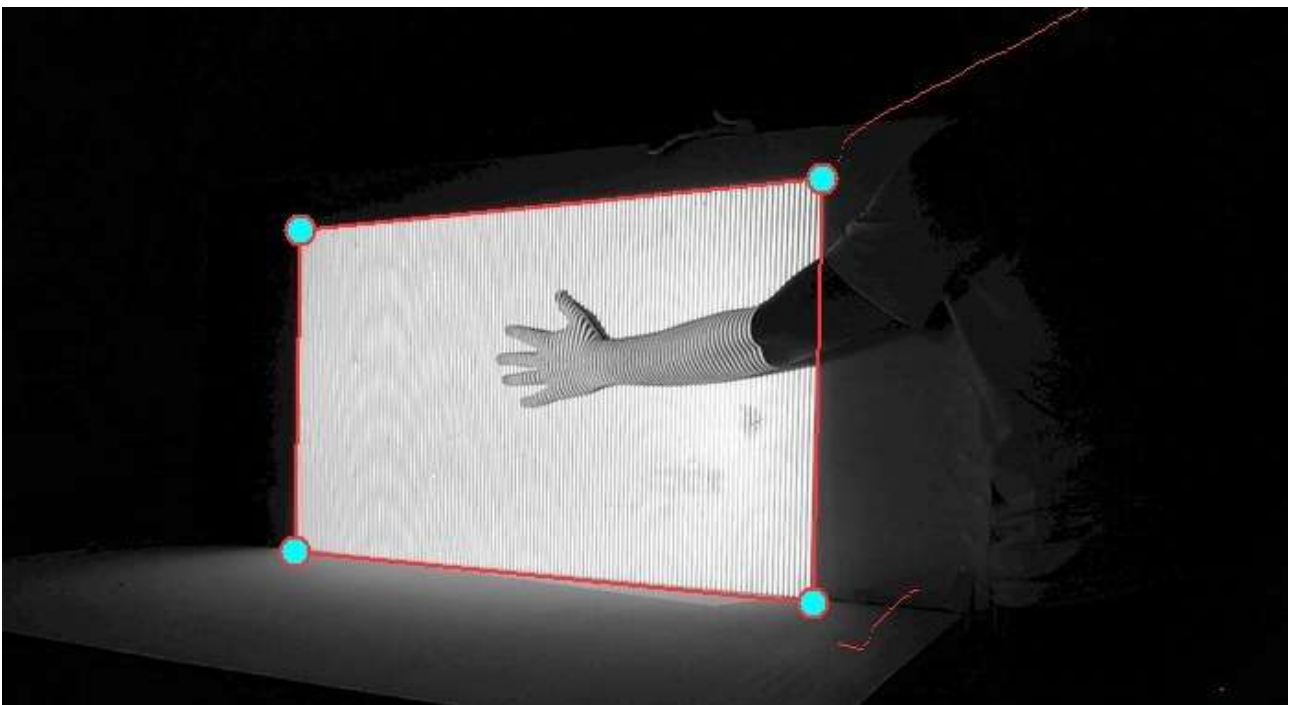
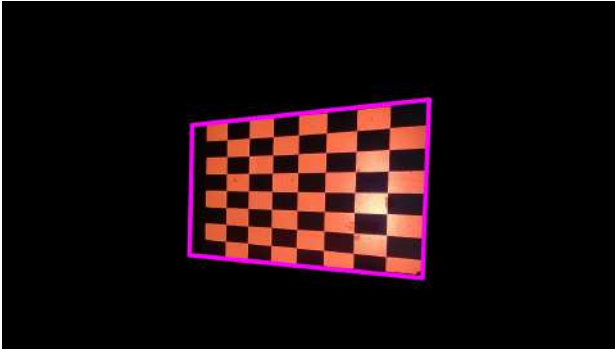
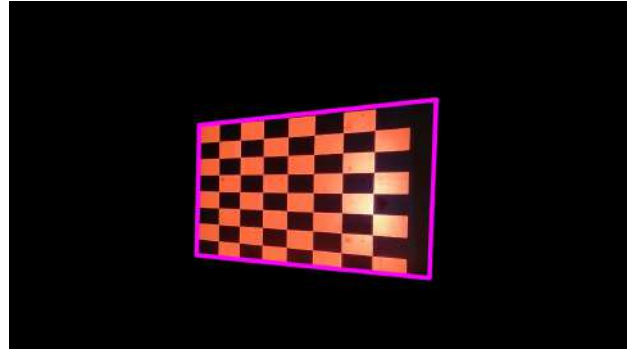


FIGURE 14 – Cache pour la mesure d'objet

Pour faciliter la mise en place du cache, nous avons choisi de le fixer lors de la calibration émetteur, de façon à ce qu'il englobe le damier pour les 2 positions z de l'écran. Ensuite nous pouvons nous permettre de garder son placement pour la mesure de l'objet car les bandes seront mesurées par la caméra à peu près au même endroit que lors de la calibration (en fonction des volumes de l'objet).



(a) Cache calibrage $z=10$



(b) Cache calibrage $z=0$

6 Intégration et validation

6.1 Interface graphique

Lors du développement de l'interface nous avons rapidement testé les fonctionnalités liées à la caméra et au projecteur, car nous avons considérés ces fonctions comme les plus importantes. Une fois que le retour caméra et la projection ont été validés nous avons pu vérifier l'enregistrement pour nous assurer que l'interface soit utilisable. Une fois l'interface fonctionnelle, nous l'avons utilisée pendant toutes les étapes suivantes du projet, ce qui nous a permis de gagner du temps sur les calibrations et les enregistrements d'images.

6.2 Validation de la calibration

Valider la calibration suppose deux étapes : vérifier dans un premier temps que les quadrillages présents sur les 4 images de calibrations sont bien détectés, et ensuite s'assurer de la pertinence des matrices de calibration.

Lors d'un essai, voici les 4 images permettant la calibration que nous avons pu prendre :

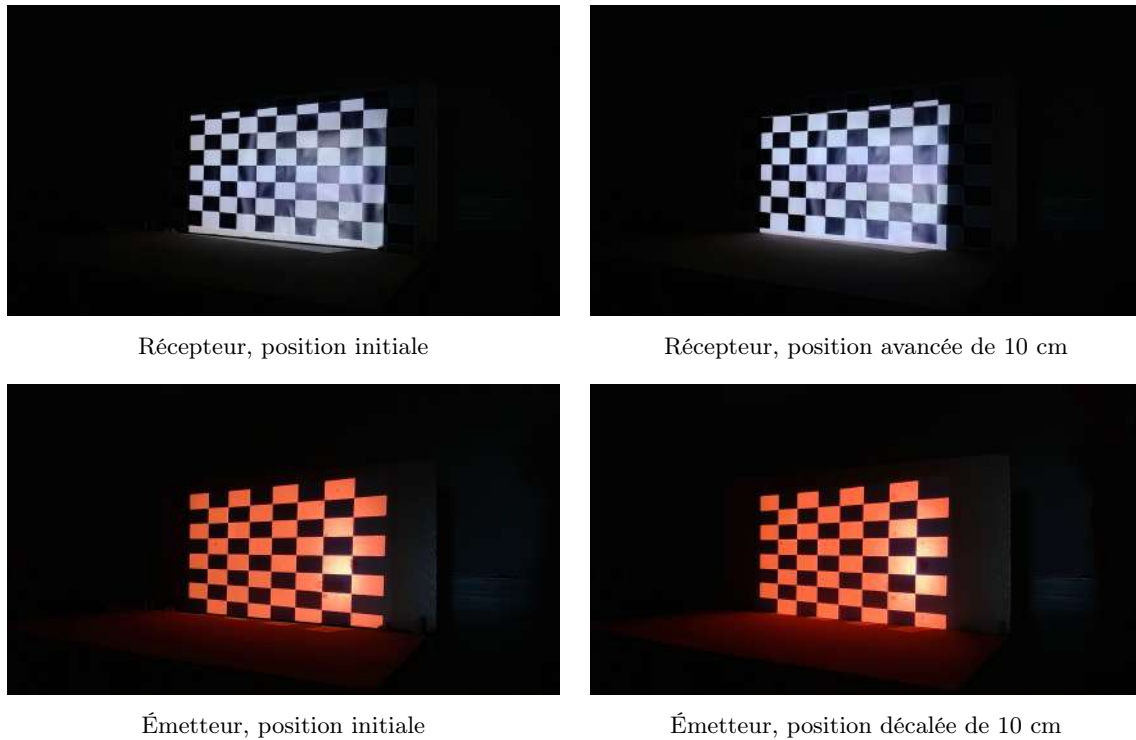


FIGURE 16 – Images brutes utilisées pour la calibration du récepteur et de l'émetteur.

À partir de ces images, l'algorithme recherche automatiquement les intersections du quadrillage. Pour contrôler cette étape, nous générons une seconde série d'images où les points détectés sont superposés aux captures. Cela permet de vérifier visuellement que le damier a bien été reconnu avant d'utiliser les coordonnées pour calculer les matrices de calibration.



FIGURE 17 – Points détectés pour la calibration du récepteur en position initiale.

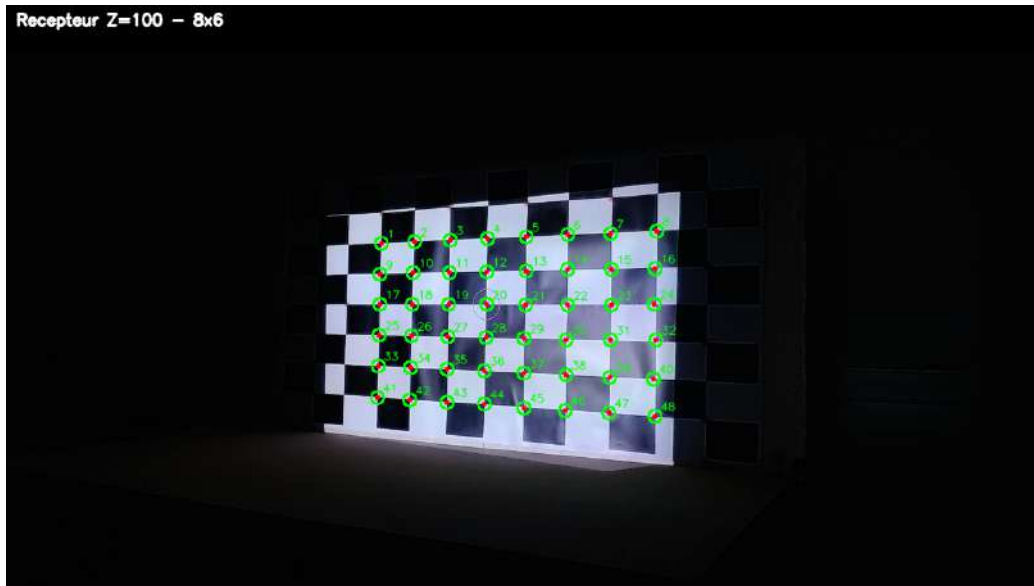


FIGURE 18 – Points détectés pour la calibration du récepteur après déplacement de 10 cm.

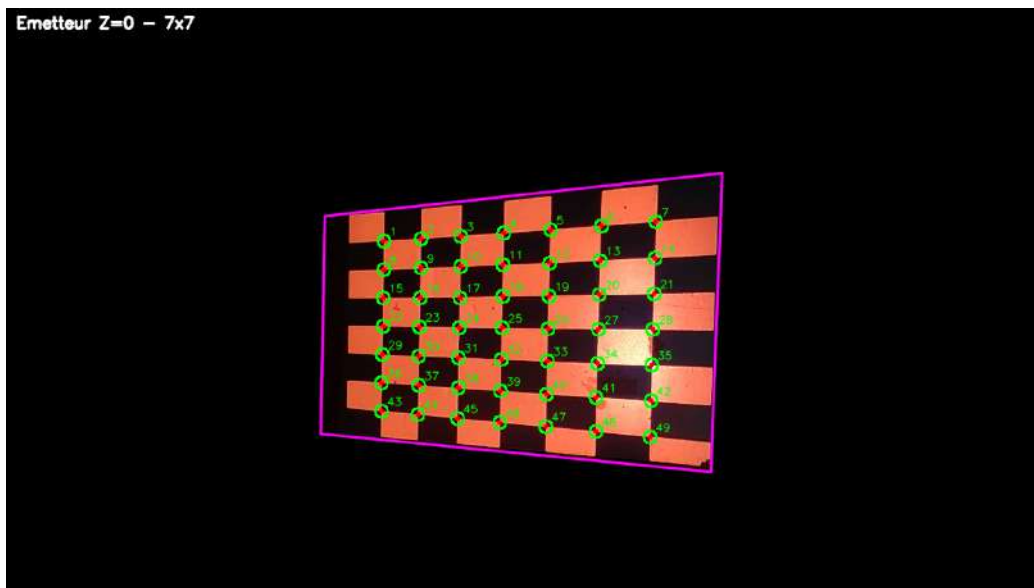


FIGURE 19 – Points détectés pour la calibration de l'émetteur en position initiale.

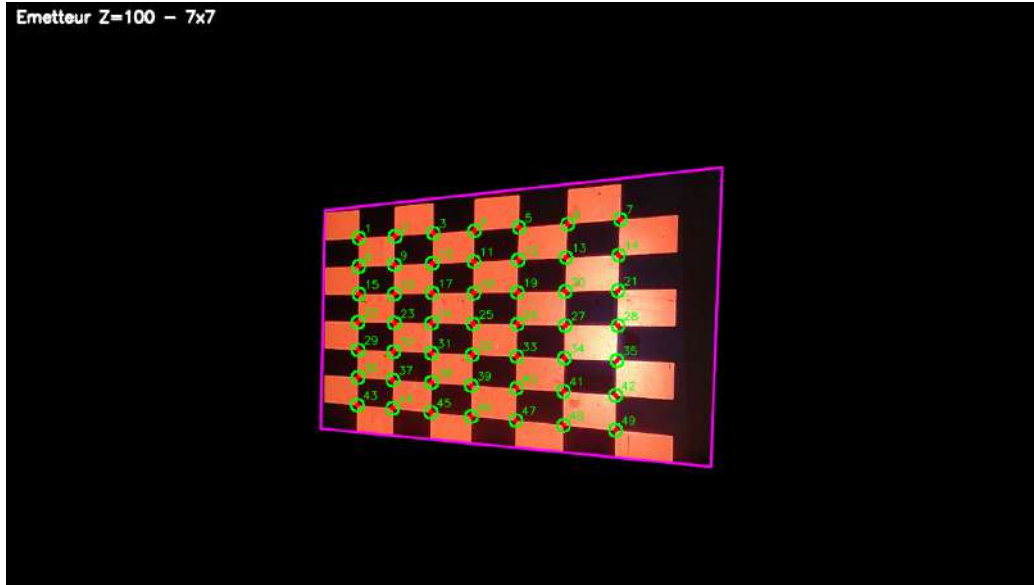


FIGURE 20 – Points détectés pour la calibration de l'émetteur après déplacement de 10 cm.

Ces images de contrôle permettent de valider la première étape de la calibration. Les points détectés doivent former un quadrillage régulier, complet et cohérent avec l'orientation du damier. Une détection exploitable se caractérise par l'absence de point manquant, par un ordre de parcours stable des intersections et par une continuité géométrique entre les lignes et les colonnes. Si l'un de ces critères n'est pas respecté, la calibration est relancée, car une seule mauvaise correspondance peut dégrader fortement les matrices calculées.

Une fois les points validés sur les quatre images, ils sont utilisés pour calculer les matrices de projection du récepteur et de l'émetteur. La pertinence de ces matrices est ensuite vérifiée par reprojection : les points 3D connus du damier sont projetés de nouveau dans les images, puis comparés aux points détectés. La validation est effectuée sur les deux positions du damier, en position initiale puis après le déplacement de 10 cm, afin de vérifier que la profondeur utilisée pour la calibration est bien prise en compte.

Pour le récepteur, les points détectés dans les images sont comparés aux projections obtenues avec la matrice M_R . Les points verts correspondent aux points détectés, les points rouges aux points reprojetés et les segments jaunes représentent l'erreur locale entre les deux positions.

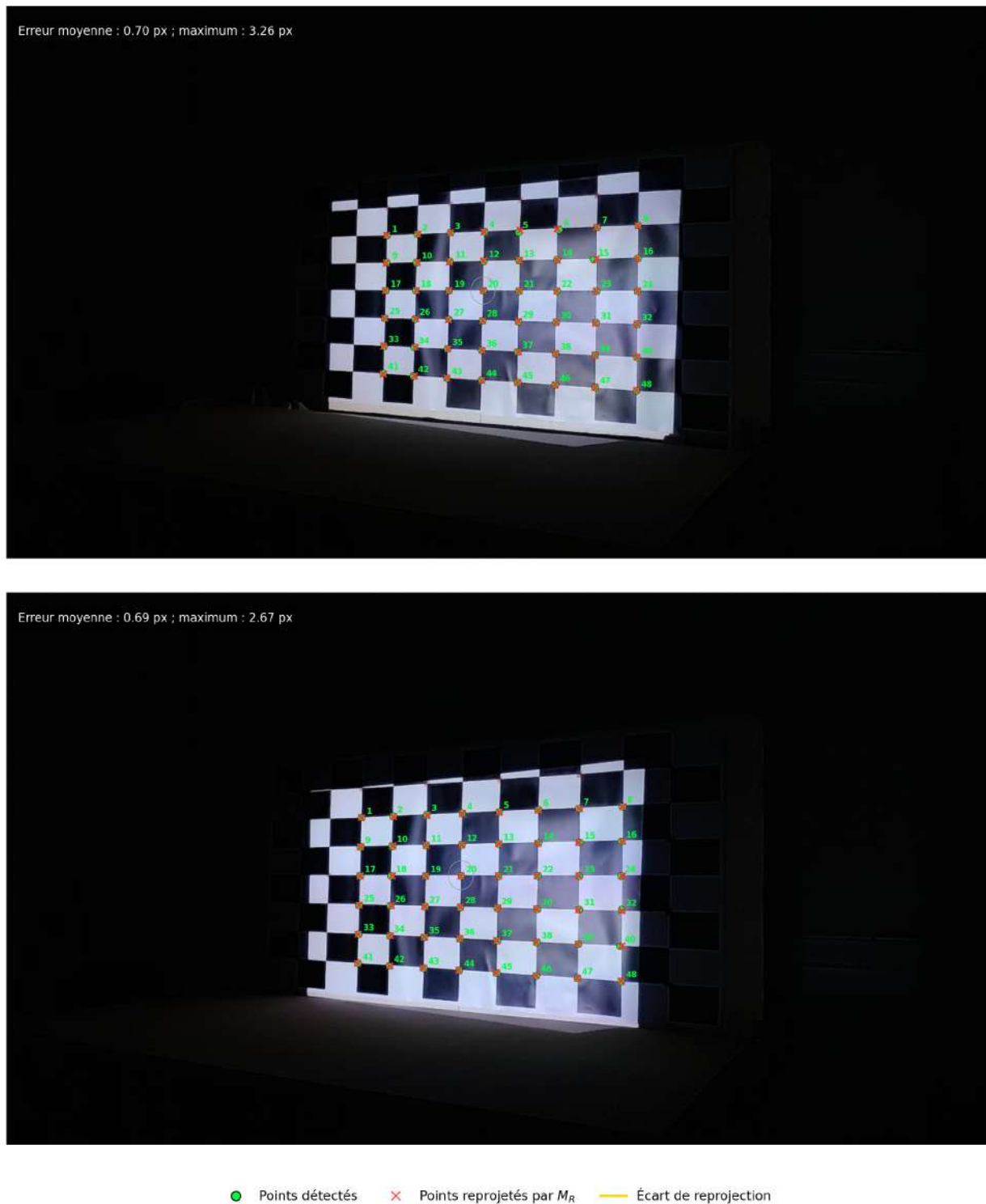


FIGURE 21 – Validation par reprojection de la matrice du récepteur sur les deux positions du damier.

Pour l'émetteur, la même vérification est menée dans le repère image du projecteur. Les coordonnées attendues du quadrillage projeté sont comparées aux coordonnées reprojetées par la matrice M_E . Cette étape valide que la calibration de l'émetteur reste cohérente avec la géométrie utilisée pour générer les franges et pour reconstruire les points 3D.

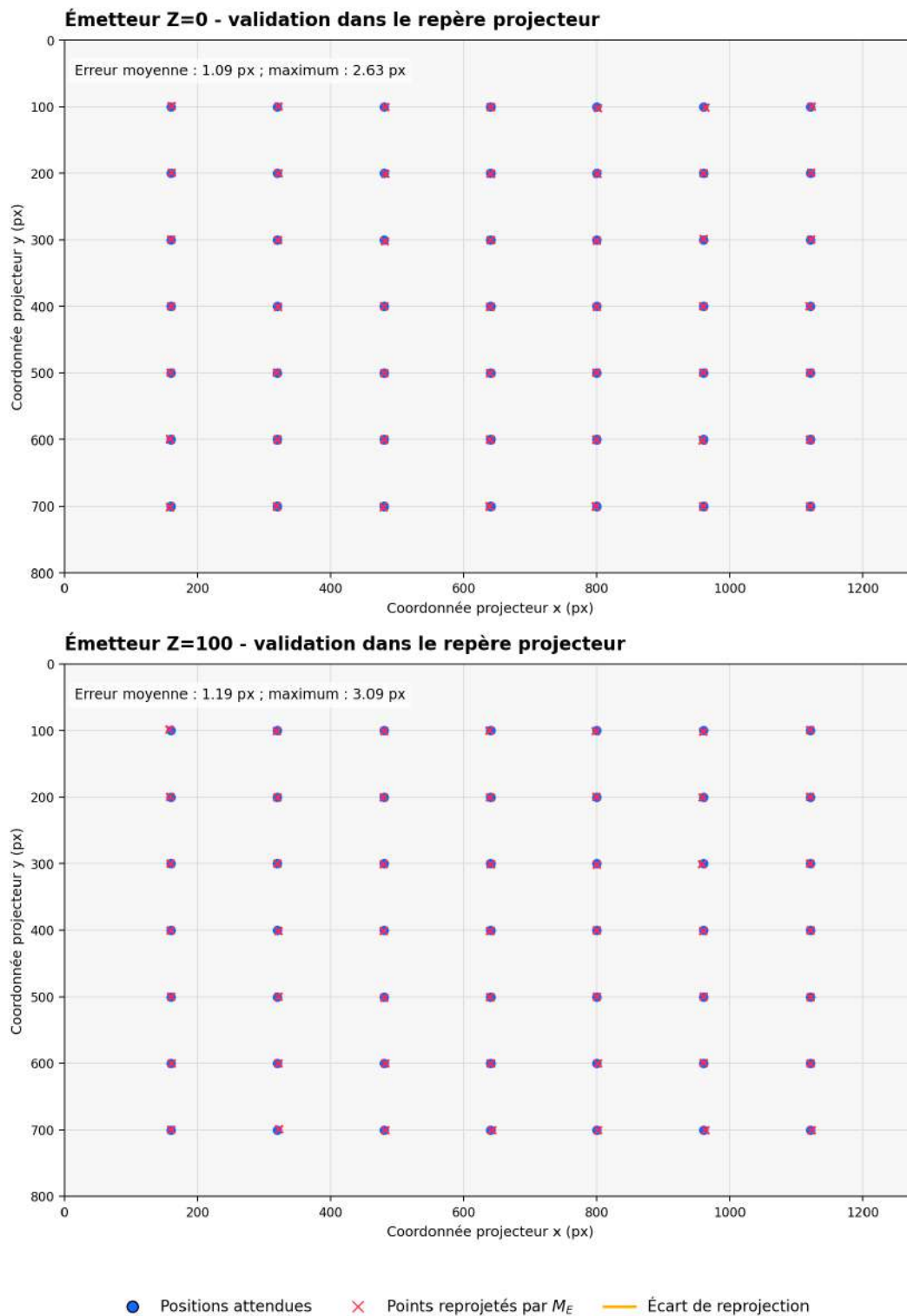


FIGURE 22 – Validation par reprojection de la matrice de l'émetteur dans le repère du projecteur.

Sur cet essai, l'erreur moyenne de reprojection est de 0,69 pixel pour le récepteur et de 1,14 pixel pour l'émetteur. Les erreurs maximales restent respectivement proches de 3,26 pixels et 3,09 pixels. Ces valeurs restent faibles à l'échelle des images et ne montrent pas de dérive globale du quadrillage. Les matrices obtenues sont donc cohérentes avec la géométrie réelle du montage et peuvent être utilisées pour la reconstruction 3D.

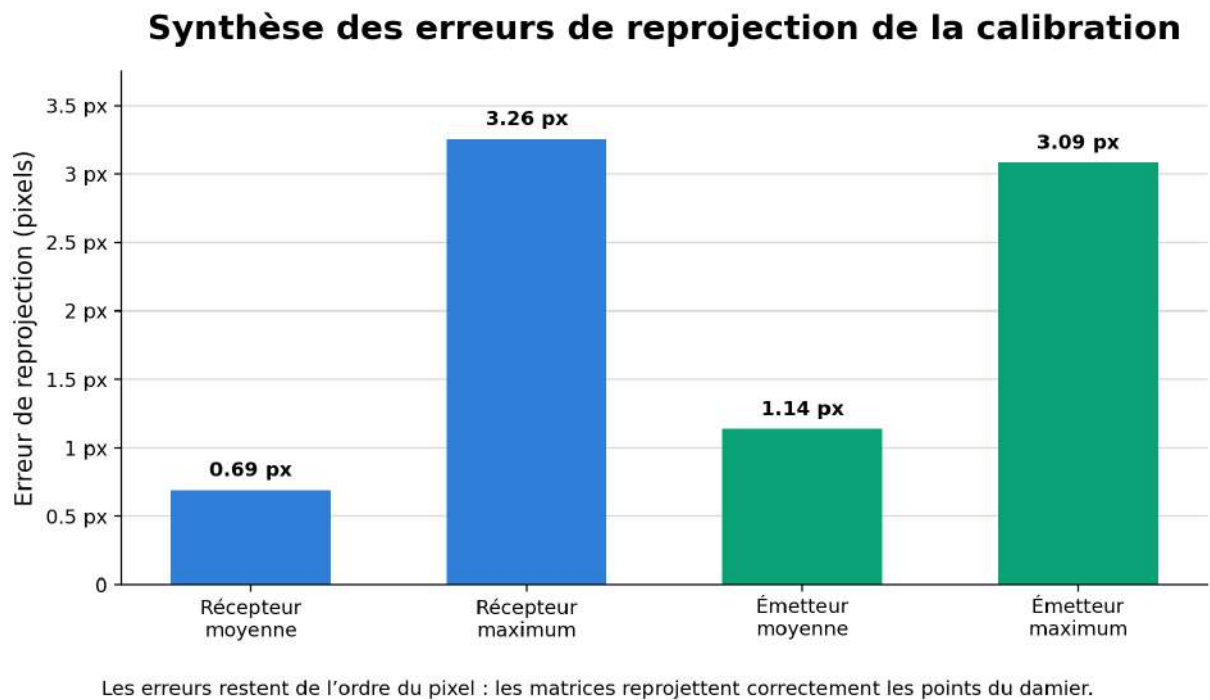


FIGURE 23 – Synthèse des erreurs de reprojection obtenues sur l'exemple de calibration validé.

6.3 Validation de la reconstruction

De même, valider la reconstruction suppose également deux étapes : vérifier que les franges sur les images de l'objet sont bien identifiées par notre algorithme, puis vérifier que l'objet reconstruit correspond bien à l'objet réel.

Voici les images brutes que nous avons obtenues pour la reconstruction de trois objets, respectivement le plan, la pyramide et le bras. Ayant choisi $N = 8$, nous avons 8 images par objet :

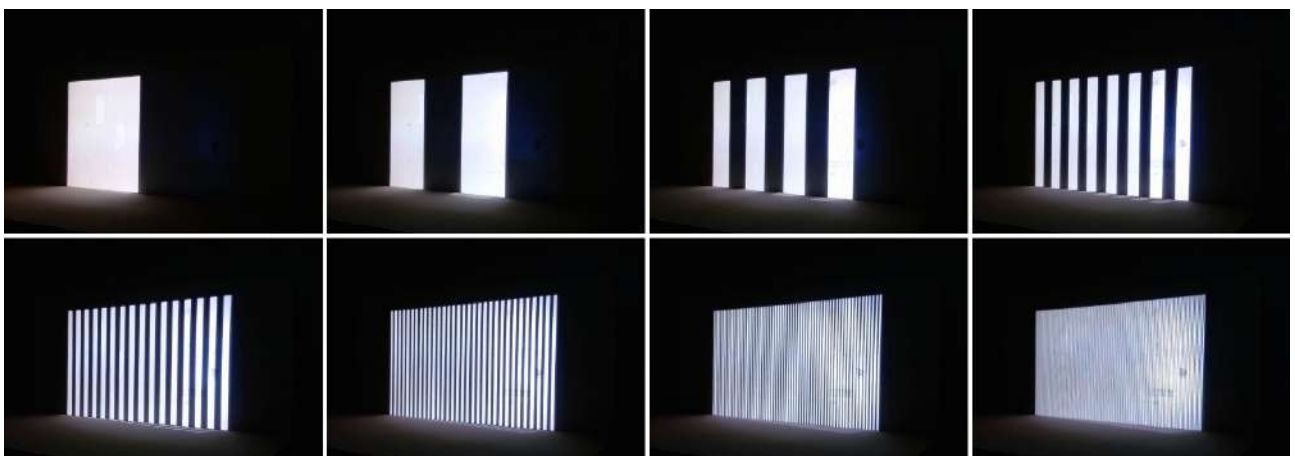


FIGURE 24 – Images brutes de reconstruction du plan

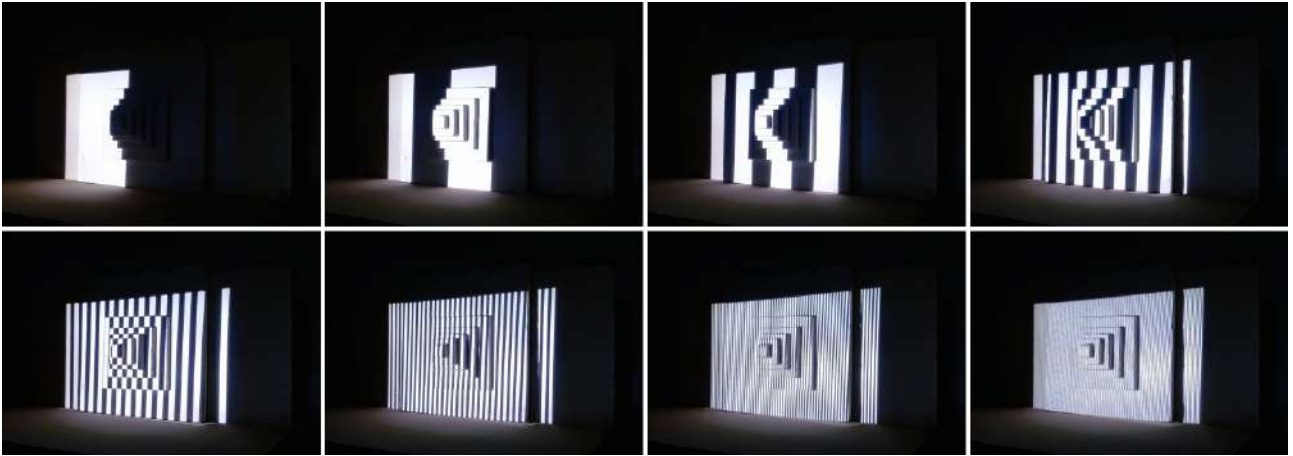


FIGURE 25 – Images brutes de reconstruction de la pyramide

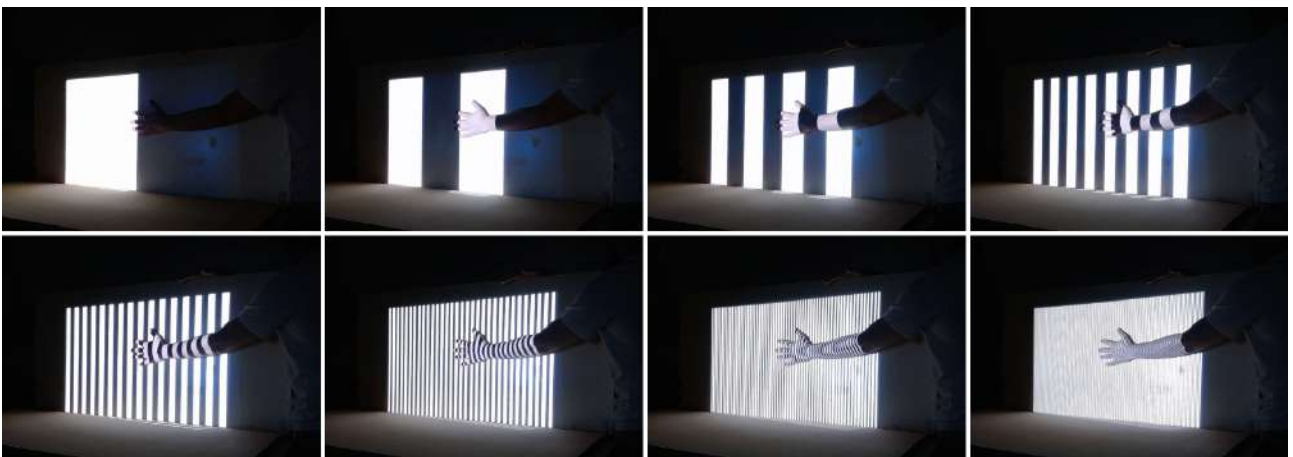


FIGURE 26 – Images brutes de reconstruction du bras

- Pour confirmer la reconnaissance des franges, nous avons utilisé une approche visuelle, en vérifiant si les franges détectées par le programme correspondent bien à celles émises par le projecteur :

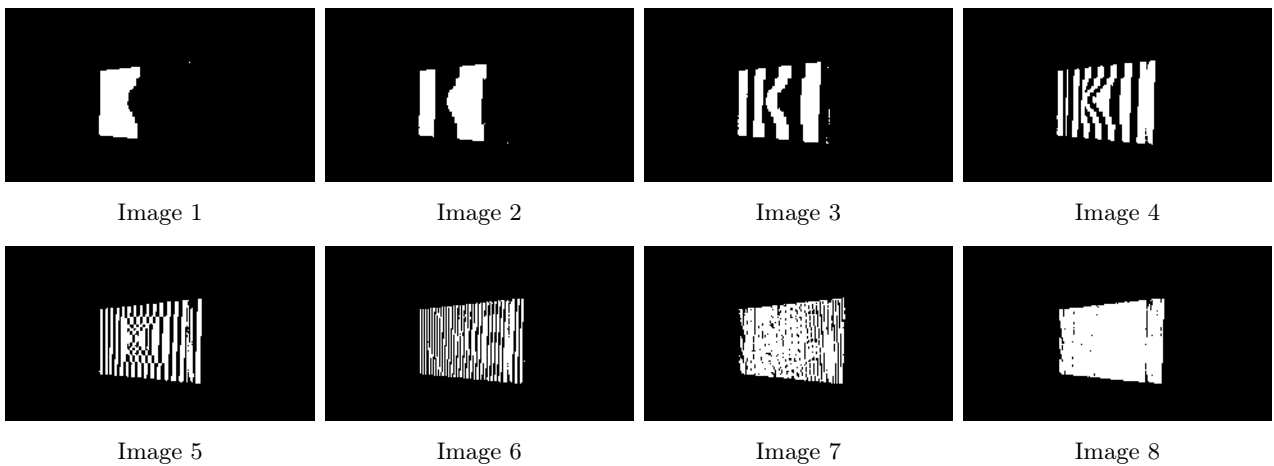


FIGURE 27 – Validation visuelle des franges détectées sur les huit images projetées.

On constate ainsi que la détection des franges est correcte pour les 7 premières images, mais beaucoup plus difficile pour la 8^e, probablement parce que la fréquence était très élevée et que les franges étaient petites sur l'image. Pour cette raison, nous avons décidé de ne retenir que les 7 premières images pour effectuer la reconstruction des objets.

- Pour confirmer que la reconstruction était fonctionnelle nous avons tenté de reconstruire différents objets parmi ceux qui nous ont été fournis. Nous avons commencé par utiliser un simple plan. Les premiers essais n'ont pas été fructueux car la reconstruction était courbée car la partie reconnaissance des points ne prenait pas les points dans le bon ordre.

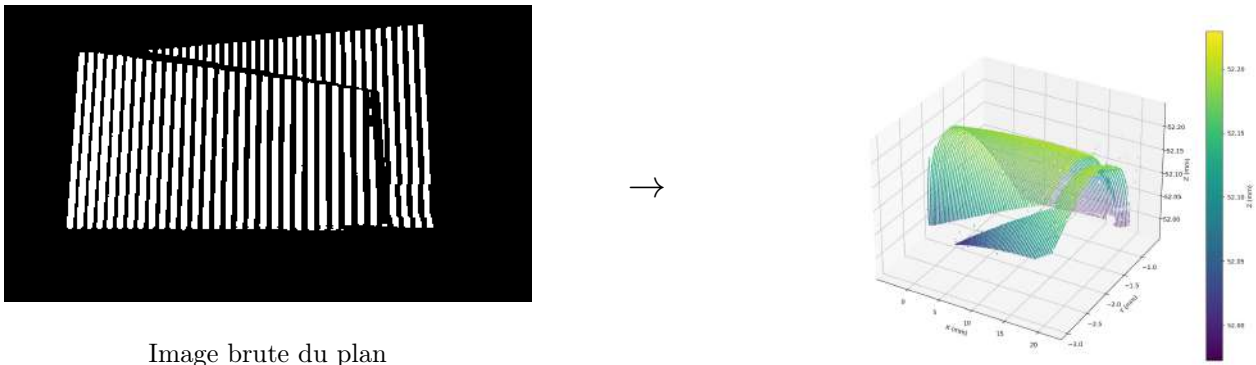


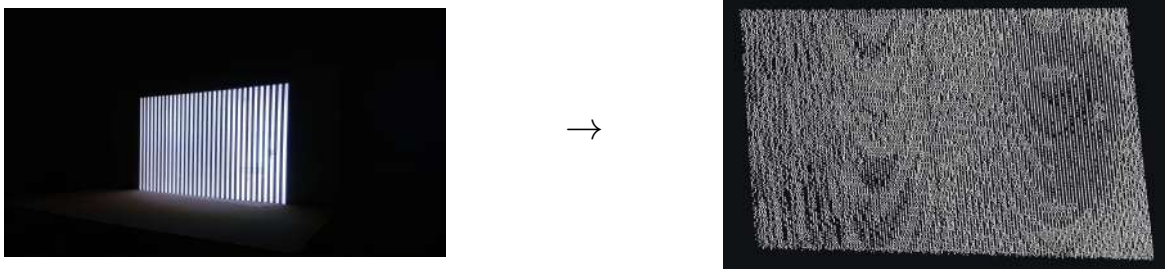
FIGURE 28 – Plan courbé

Afin de faciliter la consultation des résultats, nous avons également exporté plusieurs reconstructions sous forme de visualisations 3D interactives accessibles depuis un navigateur. Ces pages permettent de manipuler les nuages de points reconstruits, de changer l'angle de vue et de vérifier plus facilement la forme générale des objets que sur une simple capture fixe. Elles complètent donc les figures du rapport en donnant accès aux résultats reconstruits dans un format plus exploitable.

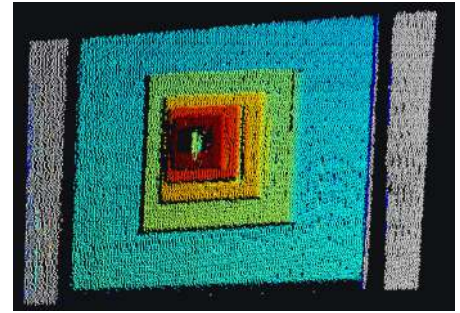
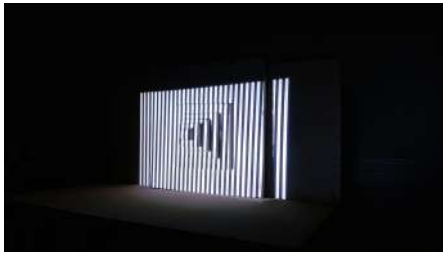
L'ensemble des visualisations est regroupé sur la page d'accueil des reconstructions 3D :

<https://math680.github.io/pronto-reconstructions-3d/>

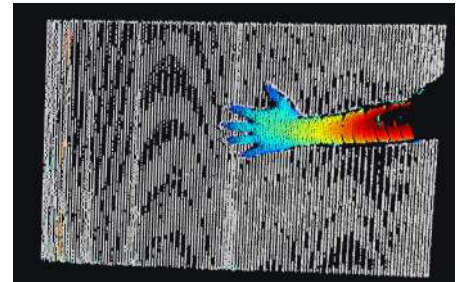
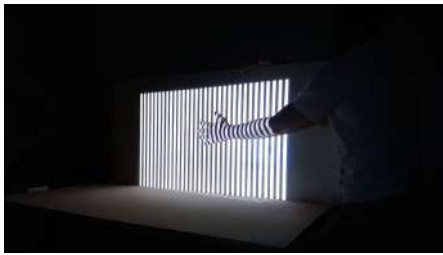
- Plan reconstruit : <https://math680.github.io/pronto-reconstructions-3d/plan.html>



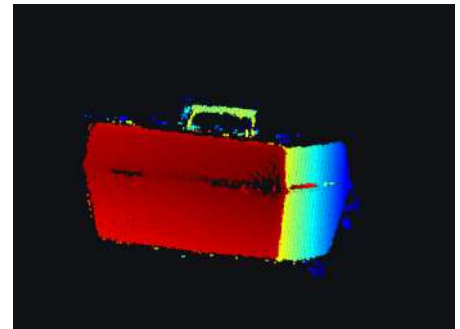
- Pyramide reconstruite : <https://math680.github.io/pronto-reconstructions-3d/pyramide-a-plat.html>



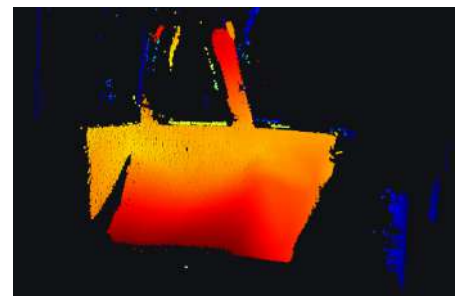
- Bras reconstruit : <https://math680.github.io/pronto-reconstructions-3d/bras.html>



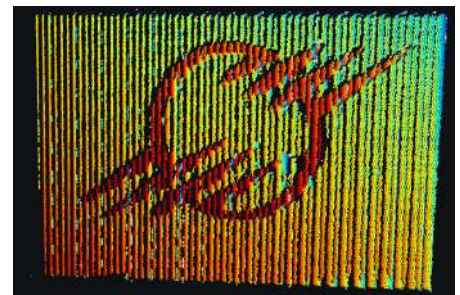
- Caisse de bricoleur reconstruite : <https://math680.github.io/pronto-reconstructions-3d/caisse-bricoleur.html>



- Sac reconstruit : <https://math680.github.io/pronto-reconstructions-3d/sac-zainab.html>



- Logo des Mines reconstruit : <https://math680.github.io/pronto-reconstructions-3d/mines.html>



Après avoir observé ces différents résultats de reconstruction, nous avons cherché à vérifier plus précisément que la déformation observée sur le plan ne se retrouvait plus sur les objets reconstruits. Pour cela, nous avons contrôlé les proportions du premier étage de la pyramide directement dans le logiciel de visualisation. La valeur de 4,6 cm indiquée sur la figure correspond à une mesure manuelle réalisée dans cet outil, utilisée comme repère relatif pour comparer les côtés entre eux, et non à la dimension physique réelle de l'objet. Nous avons mesuré la pyramide réelle : les côtés font 29,7 et 29,2 cm. Cette différence est de l'ordre de 2%, ce qui n'est pas détectable sur la modélisation à cause du changement d'échelle. La base apparaît donc carrée, ce qui est normal et confirme que les proportions sont bien conservées.

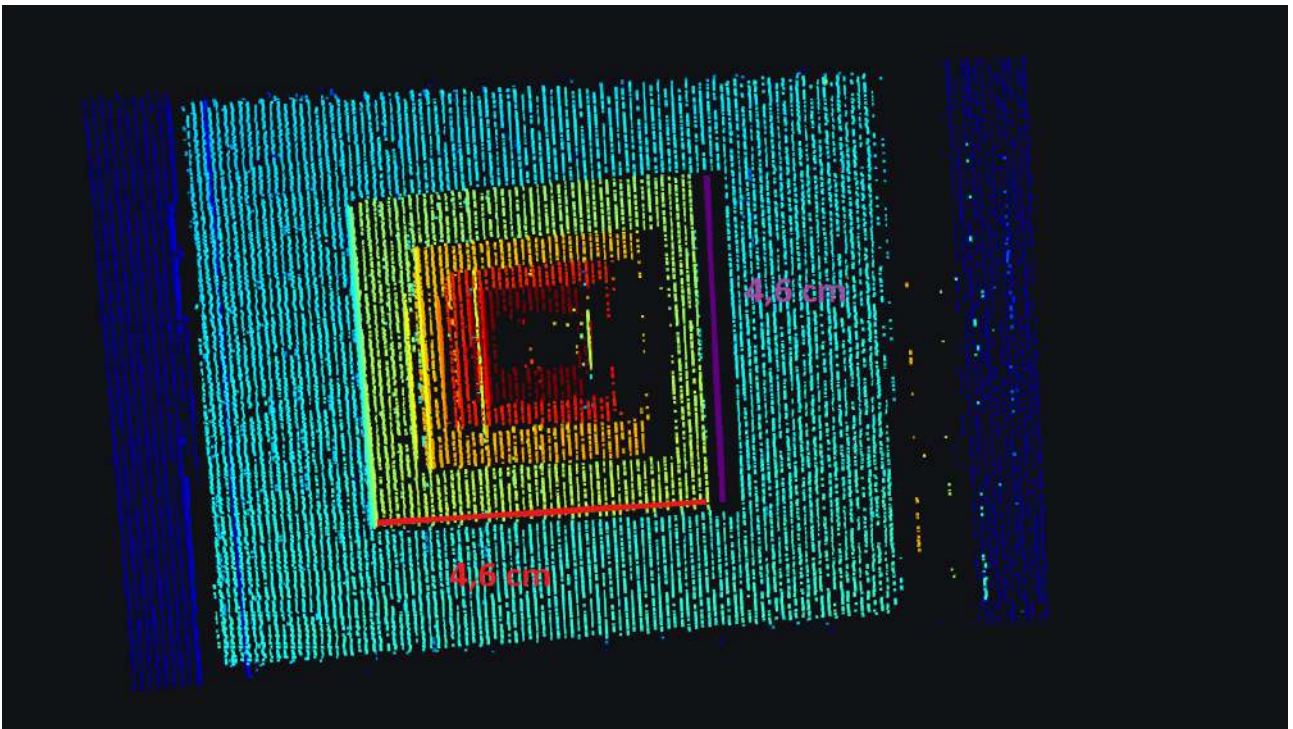


FIGURE 29 – Vérification manuelle des proportions de la pyramide reconstruite.

7 Clôture du projet

7.1 Tableau des rendus techniques/scientifiques

Livrable	Cahier des charges fonctionnel	Revue de projet	Rapport final	Recette du projet
Description	Rédigé à l'issue de la phase de cadrage, le document de cadrage rend compte de l'état initial du projet selon les différents axes : scientifique/technique et organisationnel (équipe, projet).	La revue de projet et d'équipe est réalisée à mi-parcours de la phase de production sous la forme d'une soutenance orale devant l'encadrement.	Rédigé à l'issue de la phase de production, le rapport du projet rend compte de l'état final du projet selon les différents axes : scientifique/technique et organisationnel (équipe, projet).	À l'issue du projet, la recette (réunion finale entre l'encadrant et l'équipe) rend compte de l'état final du projet selon les différents axes : scientifique/technique et organisationnel (équipe, projet).
Format	PDF	PDF	PDF	PDF
Date de rendu	dimanche 1 mars 2026, 23 :59	mardi 7 avril 2026, 23 :59	mardi 26 mai 2026, 23 :59	jeudi 28 mai 2026, 23 :59

7.2 Validation

7.2.1 Calibration

Scénario de démonstration

Objectif : Initialiser le système d'imagerie 3D et calibrer le couple caméra-projecteur pour pouvoir réaliser une acquisition par lumière structurée.

➤ Étape 1 : Installation matérielle

- Action : L'utilisateur positionne le projecteur et la caméra face à la zone de numérisation et les connecte à l'ordinateur.
- Résultat attendu : Les périphériques sont alimentés et reconnus par le système d'exploitation.

➤ Étape 2 : Lancement du système

- Action : L'utilisateur exécute le logiciel d'acquisition.
- Résultat attendu : L'interface graphique s'ouvre et affiche le flux vidéo en direct de la caméra.

➤ Étape 3 : Capture de la première image de la mire (Damier)

- Action : L'utilisateur place le damier de calibration dans la position lointaine des supports de calibration, puis clique sur le bouton "Calibration" puis "Image 1" de l'interface.

- Résultat attendu : Le logiciel fige l'image, détecte automatiquement les coins du damier et valide la capture de la première image.
- Étape 4 : Capture de la deuxième image de la mire (Damier)
 - Action : L'utilisateur place le damier de calibration dans la position proche des supports de calibration, puis clique sur le bouton "Image 2" de l'interface.
 - Résultat attendu : Le logiciel fige la deuxième image, détecte automatiquement les coins du damier et valide la capture de la deuxième image.
- Étape 5 : Capture de la première image de la mire projetée
 - Action : L'utilisateur projette le damier de calibration sur le plan une fois placé dans la position lointaine des supports de calibration, puis clique sur le bouton "Image 3" de l'interface.
 - Résultat attendu : Le logiciel fige l'image, détecte automatiquement les coins du damier projeté et valide la capture de la première image du damier projeté.
- Étape 6 : Capture de la deuxième image de la mire projetée
 - Action : L'utilisateur place le plan sur lequel est projeté le damier dans la position proche des supports de calibration, puis clique sur le bouton "Image 4" de l'interface.
 - Résultat attendu : Le logiciel fige l'image, détecte automatiquement les coins du damier projeté et valide la capture de la deuxième image du damier projeté.

Expériences de validation de la calibration

Pour valider quantitativement la précision de la calibration du couple caméra-projecteur, les protocoles expérimentaux suivants ont été mis en œuvre :

- **Expérience 1 : Évaluation de l'erreur de reprojection**
 - *Protocole* : Analyse de l'écart pixel entre les points réels détectés sur le damier et les points reprojetés mathématiquement par les matrices calculées.
 - *Critère de succès* : L'erreur de reprojection moyenne doit être strictement inférieure à 1 pixel ($\epsilon < 1$ px).
- **Expérience 2 : Mesure de fidélité métrique**
 - *Protocole* : Mesure logicielle de l'entraxe entre deux coins éloignés du damier physique (valeur nominale connue : 120 mm) à partir des données calibrées.
 - *Critère de succès* : L'écart avec la dimension réelle doit être inférieur à 0.5 mm.

7.2.2 Acquisition

Scénario de démonstration

Objectif : Scanner un objet en relief en projetant une séquence de motifs binaires afin de reconstruire sa géométrie en 3D.

- Étape 7 : Acquisition des données par code binaire

- Action : L'utilisateur laisse le plan, positionne l'objet à numériser dans la zone de travail, puis clique sur le bouton "Lancer acquisition" depuis l'interface graphique.
- Résultat attendu : Le projecteur projette automatiquement une séquence de motifs binaires (lignes noires et blanches) sur l'objet, tandis que la caméra synchronise la capture de la suite d'images.

➤ **Étape 8 : Reconstruction et traitement 3D**

- Action : L'utilisateur patiente pendant que le logiciel décode les motifs capturés et applique l'algorithme de triangulation grâce aux paramètres calculés lors de la calibration.
- Résultat attendu : Le logiciel traite les données en arrière-plan, puis l'interface graphique affiche le résultat final sous forme de nuage de points 3D.

Expériences de validation de l'acquisition

Pour évaluer les performances et la robustesse du système d'acquisition par lumière structurée (code binaire), les protocoles expérimentaux suivants ont été réalisés :

➤ **Expérience 1 : Évaluation de la fidélité géométrique**

- *Protocole* : Numérisation 3D d'un objet de géométrie et de dimensions connues avec précision (ex : un cube ou un cylindre étalon). Comparaison des dimensions extraites du nuage de points 3D avec les dimensions réelles de l'objet.
- *Critère de succès* : L'écart maximal entre les mesures reconstruites et les dimensions réelles doit être inférieur au seuil de tolérance fixé (ex : $\Delta \leq 1$ mm).

➤ **Expérience 2 : Robustesse au bruit et à la luminosité ambiante**

- *Protocole* : Répétition de la séquence d'acquisition d'un même objet sous trois conditions d'éclairage ambiant différentes (obscurité, éclairage néon standard, forte luminosité). Analyse du taux de pixels aberrants non décodés dans le nuage de points.
- *Critère de succès* : Le décodage du code binaire doit rester stable et le taux de points perdus ou aberrants doit rester inférieur à 5% dans les conditions nominales d'utilisation.

7.3 Axes d'amélioration du système

Bien que le système actuel permette une reconstruction 3D fonctionnelle, plusieurs axes d'amélioration techniques et algorithmiques peuvent être envisagés pour en augmenter les performances :

- **Optimisation des composants optiques** : Opter pour une caméra et un projecteur bénéficiant de résolutions supérieures afin d'augmenter la densité du nuage de points final et la finesse des détails reconstruits.
- **Évolution du protocole de lumière structurée** : Remplacer le codage binaire simple par une approche combinant le *Code de Gray* et le *décalage de franges sinusoïdales* (*Phase Shifting*). Cela permettrait d'atteindre une précision de reconstruction au sous-pixel près tout en réduisant la sensibilité aux bruits de transition [6,7].

- **Gestion des surfaces complexes (HDR) :** Implémenter un algorithme d'acquisition à grande dynamique (*High Dynamic Range* - HDR) en faisant varier le temps d'exposition de la caméra pour chaque motif projeté. Cela fiabiliserait la numérisation des objets présentant de fortes variations de réflectance (surfaces brillantes, métalliques ou très sombres) [9].
- **Automatisation de la reconstruction à 360° :** Intégrer un plateau tournant motorisé asservi par le logiciel d'acquisition, associé à un algorithme de recalage de nuages de points de type ICP (*Iterative Closest Point*), afin de générer automatiquement un modèle 3D complet de l'objet sans manipulation manuelle [8].
- **Automatisation de la définition du masque :** Réaliser directement dans le code la sélection du masque, plutôt que de sélectionner manuellement.

8 Retour d'expérience

8.1 Retour d'expérience sur la gestion de projet



FIGURE 30 – Gantt initial

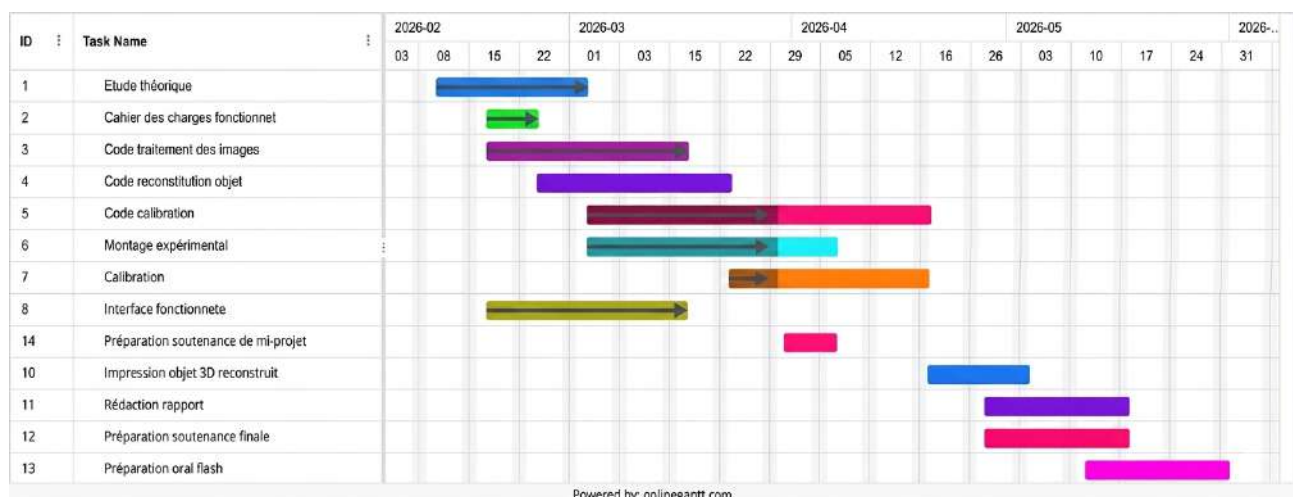


FIGURE 31 – Gantt de mi projet

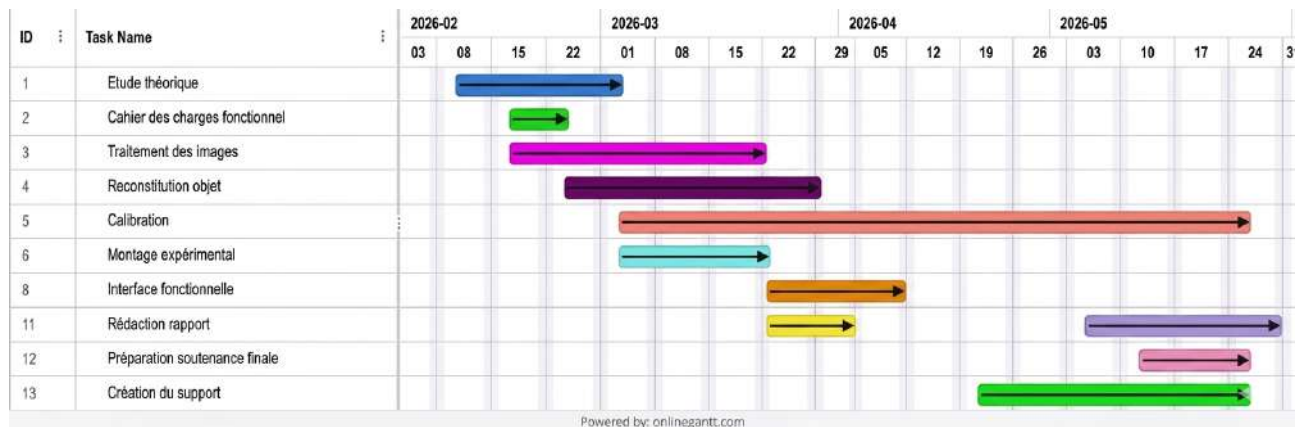


FIGURE 32 – Gantt final

Comme on peut le voir sur les images des Gantt ci-dessous, nous avons modifié notre planning à plusieurs reprises. Par exemple, la calibration a pris plus de temps que ce que nous avons prévu et le montage nous a pris moins de temps que dans nos premières estimations. Nous avons trouvé qu'il était assez difficile de définir un planning au début du projet car nous n'arrivions pas à estimer précisément le temps nécessaire pour réaliser chaque tâche. Finalement, nous avons tout de même réussi à obtenir un résultat satisfaisant dans le temps qui nous était imparti malgré le manque de précision de notre Gantt initial.

8.2 Retour d'expérience sur le travail en équipe

Nous avons collaboré à 5 sur ce projet, pour une durée d'un semestre. L'entente entre les membres était très bonne, notamment car 4 membres étaient déjà bons amis avant le projet. En effet, non seulement tout le monde s'impliquait dans le projet, mais la communication était également bonne et les avis de chacun étaient écoutés. De plus, nous avons mis en place des méthodes pour travailler efficacement en équipe : nous devons tous être présents pendant toute la durée des heures à l'emploi du temps. Pour réaliser les différentes tâches, nous avons formé des groupes de 2 ou 3 élèves selon les préférences et compétences de chacun ; et nous avons géré temporellement ces tâches avec l'utilisation d'un Gantt. Enfin, l'entraide, ainsi que l'écoute des conseils de notre tuteur, ont été primordiales pour la réussite du projet, car cela nous a permis de ne pas rester bloqués lorsque nous rencontrions des difficultés.

9 Annexes

9.1 Bibliographie

Documentations et Notices Techniques

- [1] Logitech, *C920e HD WEBCAM : Complete Setup Guide*.
- [2] Epson, *Notice technique et spécifications – Modèle 647383*.

Références de Projet et Cours

- [3] SERVAGENT N., LYS E., *Imageur 3D à code binaire*, Révisions R9 & R13, 2024.

- [4] **Équipe Projet**, *Présentation IM-3D : Spécifications et déploiement*.

Ressources théoriques

- [5] **Introduction aux principes du code binaire et structuration des données.**
- [6] **ZHANG Y., YILMAZ A.**, “Structured light based 3D scanning for specular surface by the combination of Gray code and phase shifting,” *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XLI-B3, pp. 137–142, Jul. 2016.
- [7] **CHEN T., SEIDEL H.-P., LENSCH H. P. A.**, “Modulated Phase-Shifting for 3D Scanning,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Anchorage, AK, USA, 2008, pp. 1–8.
- [8] **BESL P. J., MCKAY N. D.**, “A method for registration of 3-D shapes,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, Feb. 1992.
- [9] **ZHANG S., YAU S.-T.**, “High-dynamic-range scanning technique for 3D shape measurement,” *Optical Engineering*, Department of Mechanical Engineering (Iowa State University) & Department of Mathematics (Harvard University), vol. 48, no. 3, p. 033601, Mar. 2009.

9.2 Bonus : gestion de la couleur

En complément de la reconstruction géométrique, une fonctionnalité de colorisation du nuage de points a été ajoutée. Le principe consiste à capturer une image couleur de l’objet après la séquence de franges, puis à associer à chaque point 3D reconstruit la couleur du pixel caméra correspondant. La visualisation 3D peut ainsi afficher soit une coloration par hauteur, utile pour lire le relief, soit une coloration réaliste issue de l’image couleur.

Cette amélioration a été testée sur un bidon gris. L’image couleur sert de référence visuelle de l’objet réel, tandis que la reconstruction montre que les points 3D conservent les teintes observées par la caméra.



(a) Image couleur du bidon gris



(b) Reconstruction 3D colorisée

FIGURE 33 – Exemple de reconstruction avec association de couleur aux points 3D

9.3 Codes

Cette annexe regroupe les extraits de code les plus structurants de la version propre du projet, situés dans le dossier `Reconstruction_3D_propre`. Les parties retenues correspondent aux étapes qui portent réellement la reconstruction : calibration, décodage des franges, triangulation, orchestration du pipeline et export des visualisations. Les détails d'interface, de gestion de fichiers ou d'affichage plein écran ne sont pas repris ici afin de garder une annexe lisible.

Calcul des matrices de calibration

La calibration repose sur un calcul de matrice de projection à partir de correspondances entre des points 3D connus et leurs positions image. La matrice est estimée par résolution linéaire, puis validée par reprojection des points dans l'image.

```
1 def compute_projection_matrix(object_points, image_points):
2     point_count = len(object_points)
3     b_matrix = np.zeros((2 * point_count, 11), dtype=np.float64)
4     c_vector = np.zeros((2 * point_count, 1), dtype=np.float64)
5
6     for index, ((x, y, z), (u, v)) in enumerate(zip(object_points, image_points)):
7         row = 2 * index
8         b_matrix[row] = [x, y, z, 1.0, 0.0, 0.0, 0.0, 0.0, -u * x, -u * y, -u * z]
9         c_vector[row] = u
10
11         b_matrix[row + 1] = [0.0, 0.0, 0.0, 0.0, x, y, z, 1.0, -v * x, -v * y, -v * z]
12         c_vector[row + 1] = v
13
14     solution = np.linalg.pinv(b_matrix) @ c_vector
15     return np.append(solution.flatten(), 1.0).reshape(3, 4)
16
17 def reprojection_error(matrix, object_points, image_points):
18     homogeneous = np.hstack([object_points, np.ones((len(object_points), 1))])
19     projection = (matrix @ homogeneous.T).T
20     projection = projection[:, :2] / projection[:, 2:3]
21     errors = np.linalg.norm(projection - image_points, axis=1)
22     return float(errors.mean()), float(errors.max())
```

Listing 4 – Calcul et validation d'une matrice de projection.

Décodage des franges

Chaque pixel observé par la caméra est seuillé à partir de sa propre dynamique lumineuse. Cette approche évite d'imposer un seuil global, qui serait trop sensible aux variations d'éclairage et aux ombres. Les bits obtenus sont ensuite convertis en indice de frange projecteur.

```
1 def binariser_fringes(captures: np.ndarray, masque: np.ndarray) -> np.ndarray:
2     image_max = captures.max(axis=0).astype(np.float32)
3     image_min = captures.min(axis=0).astype(np.float32)
```

```
4     seuil = (image_max + image_min) * 0.5
5
6     binaire = (captures.astype(np.float32) > seuil[None, :, :]).astype(np.uint8)
7     binaire[:, ~masque] = 0
8     return binaire
9
10 def decoder_coordonnees_projecteur(franges_binaires: np.ndarray, masque: np.ndarray) -> np.
    ndarray:
11     franges_inversees = 1 - franges_binaires
12     nombre_bits = franges_inversees.shape[0]
13     poids = (2 ** np.arange(nombre_bits - 1, -1, -1)).astype(np.int32)[: , None, None]
14
15     code = np.sum(franges_inversees * poids, axis=0)
16     code[~masque] = 0
17     code = cv2.medianBlur(code.astype(np.uint16), 3).astype(np.float32)
18     code[~masque] = 0
19     return code
```

Listing 5 – Binarisation locale et décodage des coordonnées projecteur.

Triangulation des points 3D

Une fois le pixel caméra et l'abscisse projecteur associés, la position 3D est calculée en résolvant le système issu des matrices de projection du récepteur et de l'émetteur. Le calcul est vectorisé pour traiter tous les pixels valides du masque objet.

```
1 def trianguler(code_projecteur, masque_objet, matrice_recepteur, matrice_emetteur,
    pas_projecteur):
2     hauteur, largeur = code_projecteur.shape
3     lignes, colonnes = np.indices((hauteur, largeur), dtype=np.float64)
4     valide = masque_objet
5
6     u_recepteur = colonnes[valide]
7     v_recepteur = lignes[valide]
8     x_projecteur = (code_projecteur[valide] + 0.5) * pas_projecteur
9
10    nombre_points = len(u_recepteur)
11    systeme = np.zeros((nombre_points, 3, 3), dtype=np.float64)
12    second_membre = np.zeros((nombre_points, 3), dtype=np.float64)
13
14    systeme[:, 0, :] = u_recepteur[:, None] * matrice_recepteur[2, :3] - matrice_recepteur[0, :3]
15    systeme[:, 1, :] = v_recepteur[:, None] * matrice_recepteur[2, :3] - matrice_recepteur[1, :3]
16    systeme[:, 2, :] = x_projecteur[:, None] * matrice_emetteur[2, :3] - matrice_emetteur[0, :3]
17
18    second_membre[:, 0] = matrice_recepteur[0, 3] - u_recepteur * matrice_recepteur[2, 3]
19    second_membre[:, 1] = matrice_recepteur[1, 3] - v_recepteur * matrice_recepteur[2, 3]
```



```
20 second_membre[:, 2] = matrice_emetteur[0, 3] - x_projecteur * matrice_emetteur[2, 3]
21
22 points = np.linalg.solve(systeme, second_membre[... , None]).reshape(nombre_points, 3)
23 return points[np.isfinite(points).all(axis=1)]
```

Listing 6 – Triangulation vectorisée des points reconstruits.

Pipeline de reconstruction

Le pipeline principal enchaîne les étapes précédentes : chargement des captures, application du masque, décodage des franges, triangulation, filtrage, changement de repère et sauvegarde des résultats. C'est ce bloc qui rend la reconstruction reproductible pour plusieurs objets.

```
1 def reconstruire_un_dossier(nom_objet, matrice_recepteur, matrice_emetteur,
2                             centre_plan, axes_plan, filtrer_sous_plan, tolerance_sous_plan):
3     dossier_capture = DOSSIER_CAPTURES / nom_objet
4     nombre_captures = compter_captures_utilisables(dossier_capture)
5     captures = charger_captures(dossier_capture, nombre_captures)
6     masque, polygone = charger_masque(dossier_capture, captures[0].shape)
7     captures = appliquer_masque(captures, masque)
8
9     franges_binaires = binariser_franges(captures, masque)
10    code_projecteur = decoder_coordonnees_projecteur(franges_binaires, masque)
11    masque_objet = masque_objet_depuis_franges(captures, masque)
12    pas_projecteur = LARGEUR_PROJECTEUR_PX / float(2**nombre_captures)
13
14    points = trianguler(code_projecteur, masque_objet, matrice_recepteur, matrice_emetteur,
15                        pas_projecteur)
16    mode_plan = nom_objet.lower() == "plan"
17    points = filtrer_plan(points) if mode_plan else filtrer_objet(points)
18
19    points_affiches = points
20    if centre_plan is not None and axes_plan is not None:
21        points_affiches = passer_dans_repere_plan(points, centre_plan, axes_plan)
22        if not mode_plan and filtrer_sous_plan:
23            garder = points_affiches[:, 2] >= -tolerance_sous_plan
24            points = points[garder]
25            points_affiches = points_affiches[garder]
26
27    dossier_sortie = DOSSIER_RESULTATS / nettoyer_nom(nom_objet)
28    dossier_sortie.mkdir(parents=True, exist_ok=True)
29    sauver_html_3d(dossier_sortie / "vue_3d_interactive.html", nom_objet, points_affiches)
30    np.savetxt(dossier_sortie / "points_3d_repere_plan.txt", points_affiches, fmt="%.6f")
```

Listing 7 – Enchaînement principal de la reconstruction d'un objet.

Visualisation interactive

Pour faciliter l'analyse des résultats, les nuages de points sont exportés sous forme de pages HTML interactives. Le code ci-dessous réduit le nombre de points si nécessaire, centre le nuage et prépare les données qui seront affichées dans le navigateur.

```
1 def sauver_html_3d(chemin: Path, titre: str, points: np.ndarray) -> None:
2     points_affiches = points
3     if len(points_affiches) > 70000:
4         indices = np.linspace(0, len(points_affiches) - 1, 70000, dtype=int)
5         points_affiches = points_affiches[indices]
6
7     points_centres = points_affiches - points_affiches.mean(axis=0)
8     echelle = float(np.max(np.ptp(points_centres, axis=0)))
9     z_min, z_max = np.percentile(points_centres[:, 2], [1, 99])
10
11     donnees = [
12         [round(float(x), 3), round(float(y), 3), round(float(z), 3)]
13         for x, y, z in points_centres
14     ]
15
16     # Le HTML complet contient le canvas et les controles souris/molette.
17     html = "<html> ... canvas interactif ... </html>"
18     chemin.write_text(html, encoding="utf-8")
```

Listing 8 – Préparation d'une visualisation 3D interactive.

9.4 Retours d'expériences individuels

Boulard Gaëtan

Le projet fut très instructif, j'ai beaucoup appris sur le plan technique et humain. En effet, ce projet m'a demandé d'acquérir des compétences scientifiques (particulièrement en mathématiques) de pointe, telle qu'une compréhension avancée des matrices de projection. Mais c'est l'ambiance globale du groupe qui m'a le plus marqué, j'ai été agréablement surpris de la dévotion de mes camarades en ce projet, j'ai rarement vu un groupe où tout le monde travaille de manière équivalente et où la parole est à ce point libre. Globalement un très bon groupe, je serais ravi de faire un nouveau projet avec eux.

Nous avons rencontré un problème pendant la phase de calibration, nous déplaçons le damier à la main. C'était beaucoup trop approximatif et cela faussait la précision de nos mesures. Pour régler ce problème, j'ai conçu et imprimé en 3D un système de supports spécifiques. L'idée est d'en placer deux de chaque côté du damier pour bien le caler. J'ai configuré ces supports avec deux positions fixes, une arrière et une avant, qui bloquent complètement les mouvements de rotation ou les écarts sur les côtés. Cela permet de faire glisser le damier d'une position à l'autre pour obtenir une translation parfaite de 10 centimètres vers l'avant. Grâce à cela, nous avons supprimé l'erreur humaine : le mouvement est devenu 100% répétable et le déplacement lors de la calibration est maintenant d'une précision parfaite.

Jullière Thomas

Ce projet était un bon exercice de travail en équipe sur un projet plutôt complexe. Il m'a permis de développer mes capacités de travail en équipe, et le côté informatique était également intéressant avec la manipulation de divers logiciels.

Rannou Mathieu

Ce projet m'a permis de découvrir les difficultés concrètes liées au développement d'un système de reconstruction 3D complet, depuis la calibration jusqu'à la visualisation finale des résultats. J'ai principalement travaillé sur les aspects logiciels et sur l'intégration des différentes parties du projet afin d'obtenir une chaîne de reconstruction fonctionnelle et simple à utiliser.

Le développement de l'interface graphique et l'automatisation des acquisitions m'ont permis de mieux comprendre l'importance de l'ergonomie et de l'organisation du code dans un projet expérimental. J'ai également appris à gérer des problèmes pratiques liés au traitement d'image, à la calibration et à la qualité des reconstructions obtenues.

Enfin, ce projet a été une bonne expérience de travail en équipe, notamment pendant les phases d'intégration où il fallait faire fonctionner ensemble des éléments développés séparément.

Cordier Léandre

Le projet m'a beaucoup appris sur comment travailler en équipe et comment produire des solutions techniques à des problèmes. J'ai appris une méthode d'imagerie 3D grâce à ce projet et j'ai pu approfondir le sujet en en débattant avec mes camarades. Globalement, notre groupe était ouvert au débat pour réaliser au mieux nos objectifs et motivé à travailler. Ce climat nous a permis d'avancer correctement et d'arriver à un résultat satisfaisant. J'ai beaucoup apprécié l'efficacité que nous avons eu dans la répartition du travail puisque nous avons été capables d'avancer sur deux tâches simultanément à diverses étapes du projet en divisant notre équipe en deux groupes selon les aptitudes de chacun.

Pendant le projet, l'importance de l'organisation nous a souvent été soulignée par notre encadrant, qui nous a notamment conseillé de faire un diagramme de Gantt pour fixer des échéances dans ce travail. J'ai choisi de m'occuper de cette tâche car elle est liée au rôle de supervision que j'ai depuis le contrat d'équipe. J'ai ainsi repris le diagramme que nous avons fait pour le premier rendu pour le compléter, l'adapter et donner une meilleure idée à chacun de ce qui devait être fait et sur quelle période de temps. Cette amélioration de l'organisation a permis au groupe de gagner du temps.

Ramsis Zainab

Sur le plan collaboratif, ce projet a renforcé mon expérience du travail en équipe, notamment lors des phases critiques d'intégration de modules développés séparément.

Sur le plan technique, l'enseignement majeur réside dans la gestion de l'écart entre théorie et pratique. Si la maîtrise des modèles mathématiques de reconstruction laissait présager une progression fluide, la confrontation

aux données réelles a fait émerger des verrous expérimentaux complexes et imprévisibles. Cette expérience démontre les limites d'une approche purement séquentielle. Pour de futurs projets d'ingénierie similaires, il apparaît indispensable de mener le développement théorique et les tests empiriques en parallèle, selon un cycle itératif, afin d'identifier et de corriger ces divergences le plus tôt possible.